

Manual Técnico — Mostre Seu Talento ♦ Friendapp

CAMADA 1 — ARQUITETURA DE ENTRADA E ORQUESTRAÇÃO (GATEWAY & AUTH)

1. Propósito da Camada

A Camada 1 define **todo o ponto de entrada** do sistema “Mostre Seu Talento” (MST).

Ela garante que:

- somente usuários autenticados acessem o MST,
- cada requisição siga regras de segurança,
- a validação de dados seja consistente,
- e que todo fluxo seja roteado corretamente para os microserviços internos.

É a porta de entrada da funcionalidade.

2. Responsabilidades Técnicas

A camada é responsável por:

2.1 Autenticação

- Aceitar tokens JWT válidos.
- Verificar escopos adequados (ex: `mst.create`, `mst.view`).
- Verificar expiração dos tokens.
- Negar acesso sem autenticação.

2.2 Validação de Schema

- Validar payloads de criação e publicação de conteúdo.
- Garantir formatos consistentes:
 - strings $\leq$ limite
 - JSON válido
 - tipos corretos
 - campos obrigatórios presentes

2.3 Rate Limit

- Limitar requisições por usuário/IP.
- Proteger contra spam de uploads.

2.4 Orquestração

Roteia requisições para:

- talent-engine
- ingestion-worker
- feed-service
- wallet-service
- moderation-service

3. Endpoints Governados pela Camada

3.1 POST /mst/create

Inicia a criação de conteúdo.

Entrada:

```
{
  "creator_id": "uuid",
  "title": "string",
  "type": "video|audio|text|mixed",
  "intent": "string",
  "language": "pt-BR"
```

```
}
```

Saída:

```
{
  "temp_id": "uuid",
  "upload_urls": [
    {"part": 1, "url": "https://..."}
  ]
}
```

3.2 POST /mst/commit

Finaliza a criação e dispara a ingestão.

Entrada:

```
{
  "temp_id": "uuid",
  "media_refs": [...],
  "transcription": "string"
}
```

Saída:

```
{
  "talent_id": "uuid",
  "status": "pending"
}
```

3.3 GET /mst/{id}

Retorna dados do conteúdo.

4. Fluxo de Execução (Pseudocódigo)

```
onRequest(req):  
  token = req.headers["Authorization"]  
  
  if not validate_jwt(token):  
    return 401  
  
  if rate_limit_exceeded(token.user):  
    return 429  
  
  body = validate_schema(req.body)  
  if not body.valid:  
    return 400  
  
  return route_to_service(req)
```

5. Retentativas e Tolerância a Falhas

- Timeout por upstream: **3 segundos**
- Retentativa exponencial: **máx. 3 tentativas**
- Retentativas apenas em operações idempotentes
- Erro upstream → retornar **502 Bad Gateway**

6. Logs Gerados

Formato JSON estrito:

```
{  
  "request_id": "uuid",  
  "user_id": "uuid",  
  "path": "/mst/create",  
  "status": 200,  
  "latency_ms": 132,  
  "upstream_service": "talent-engine",
```

```
"upstream_status": 200
}
```

Gravados em:

- ELK Stack
- Prometheus (latência)
- OpenTelemetry (tracing)

7. Métricas e SLOs

- **Latência p95 GET** < 300ms
- **Latência p95 POST** < 500ms
- **Taxa de falha autorizada** < 0.1%
- **Tempo de disponibilidade:** 99.95%

8. Testes Exigidos

- Unit Tests (validação, autenticação)
- Integration Tests com mock de `talent-engine`
- Load tests (upload burst)
- Teste de segurança (JWT inválido, ataque de replay)

9. Encerramento da Camada 1

A Camada 1 estabelece a **fundação de segurança e entrada** do MST.

Nenhuma operação do sistema ocorre sem passar por ela.

É o checkpoint central que protege, valida e direciona todo o fluxo.

CAMADA 2 — PIPELINE DE INGESTÃO E NORMALIZAÇÃO DE MÍDIA (INGESTION-

WORKER)

1. Propósito da Camada

A Camada 2 é responsável por **processar todo o conteúdo bruto** enviado pelo usuário:

- vídeo
- áudio
- texto (quando houver)
- thumbnails
- frames
- metadados

Ela transforma a mídia original em um formato **padronizado, leve, indexável e pronto para IA**.

É o “liquidificador técnico” que prepara tudo para a Aurah-Core analisar.

2. Componentes Internos

- `ingestion-worker` (Kubernetes Job/Deployment)
 - `ffmpeg` (transcodificação)
 - `whisper-service` (ASR – transcrição)
 - `vision-lite` (detector leve de imagens)
 - `s3-client` (upload final)
-

3. Fluxo Completo da Camada (Passo a Passo)

3.1 Receber evento de commit

Evento recebido de:

mst.commit → Kafka topic: talent.ingest.request

Payload:

```
{
  "temp_id": "uuid",
  "creator_id": "uuid",
  "media_refs": [...],
  "intent": "string",
  "language": "pt-BR"
}
```

3.2 Download da mídia

O worker baixa os arquivos via URLs temporárias (presigned URLs).

Regras:

- timeout máx 15s
- tamanho máx 250MB
- se falhar 3 vezes → erro de ingestão

3.3 Validação do Contêiner

Aceita:

- MP4
- MOV
- WebM

Rejeita:

- MKV
- AVI
- formatos fragmentados

Se inválido → marcar `error_invalid_container`

3.4 Transcodificação

Padroniza o conteúdo para um formato único:

- **Vídeo:**
 - codec: H.264
 - resolução: 720p
 - fps: 30
 - bitrate: 2 Mbps
- **Áudio:**
 - PCM 16-bit
 - 16 kHz
 - mono

Comando exemplo:

```
ffmpeg -i input.mp4 -vf "scale=1280:720" -r 30 -c:v libx264 -preset faster  
-c:a pcm_s16le output.mp4
```

3.5 Extração de Frames

Extrai 1 frame por segundo:

```
ffmpeg -i video.mp4 -vf fps=1 frame_%03d.jpg
```

Frames usados para:

- análise de luz
- detecção de rosto
- heurísticas visuais
- thumbnails automáticos

3.6 Geração de Thumbnails

3 tamanhos:

- 120px
- 480px
- 720px

Arquivo salvo em S3:

```
thumb_small.jpg
thumb_medium.jpg
thumb_large.jpg
```

3.7 Análise visual leve (vision-lite)

Executa 3 detectores rápidos:

- `face_presence` (0/1)
- `motion_magnitude` (média do optical flow)
- `brightness_level` (0–1)

Armazena em `visual_heuristics` no metadata.

3.8 ASR – Transcrição

Se o usuário não enviou a transcrição manual:

```
transcription = whisper(audio_path)
confidence = whisper_confidence
```

Se falhar:

- marcar `needs_transcription = true`
- IA continua sem texto (fallback da camada 12)

3.9 Geração de Metadata Final

Exemplo:

```
{
  "duration": 37.2,
  "size_mb": 12.4,
  "resolution": "720p",
  "has_face": true,
  "motion_level": 0.42,
  "brightness": 0.63,
  "transcription_confidence": 0.91
}
```

3.10 Upload Final para S3

Estrutura:

```
s3://friendapp/mst/{creator_id}/{talent_id}/processed720p.mp4
s3://.../frames/frame_001.jpg
s3://.../thumbnails/thumb_small.jpg
```

3.11 Persistência no Banco

Registro inicial em `talent_items` :

- status = `ingested`
- metadata salvo
- media_refs substituídos pelos paths padronizados

3.12 Emissão do evento de saída

Evento:

`talent.ingested`

Payload:

```
{
  "talent_id": "uuid",
  "creator_id": "uuid",
```

```
"media_refs": [...],  
"metadata": {...}  
}
```

Esse evento ativa a Camada 3 (inferência da IA).

4. Regras de Erro (Failover)

- Falha no download → 3 tentativas → marcar `error_download`
 - Falha no ffmpeg → marcar `error_transcoding`
 - Falha no ASR → `needs_transcription = true`
 - Falha no upload → retentar 5x com backoff
-

5. Logs

Formato:

```
{  
  "talent_id": "uuid",  
  "step": "transcoding",  
  "status": "ok",  
  "latency_ms": 4872  
}
```

6. Métricas

- `ingest_time_p95` < 7s
 - `transcoding_success_rate` > 98%
 - `thumbnail_generation_rate` > 99.5%
-

7. Encerramento da Camada 2

A Camada 2 garante que o conteúdo bruto seja transformado em um formato **perfeito para IA, feed e moderação**.

Nada segue adiante sem passar por essa etapa.

CAMADA 3 — PIPELINE DE INFERÊNCIA MULTIMODAL (AURAH-CORE)

1. Propósito da Camada

A Camada 3 executa **toda a análise de IA** do conteúdo no MST.

É aqui que o sistema extrai:

- Vetores de áudio
- Vetores de vídeo
- Vetores de texto
- Vetor multimodal fusionado
- ImpactScore final
- RiskScore
- Archetype Scores
- Confidence Score
- Explanations (explicabilidade técnica)

Essa camada alimenta:

- Feed (ranking)
- Moderação
- Seleção de talentos
- Relevância
- Curadoria

- Eventos

Ela é o cérebro técnico do MST.

2. Arquitetura do Aurah-Core (Visão Técnica)

A arquitetura é composta por 4 torres de análise + 1 módulo de fusão:

2.1 Audio Tower

- MFCC 13 coeficientes
- Spectral Centroid
- Zero Crossing Rate
- Pitch Variation
- Prosody Vector (8d)

Saída: **vetor de 32 dimensões**

2.2 Video Tower

Técnicas usadas:

- CNN ResNet-34 (frames)
- Face embedding (se houver rosto)
- Expression recognition (feliz, neutro, triste, surpresa, raiva)
- Optical Flow Magnitude (movimento)
- Lighting normalization

Saída: **vetor de 48 dimensões**

2.3 Text Tower

Se houver transcrição:

- Embedding via modelo transformer (ex: MiniLM)
- PCA → redução para 32 dimensões
- Sentiment Score (-1 a 1)

- Emotion Probabilities (5 dimensões)

Saída final: **vetor de 32 dimensões**

Se não houver texto, fallback:

- vetor textual = zeros
- pesos são renormalizados na Camada 12

2.4 Crossmodal Tower

Combina:

- Lip-sync alignment (0-1)
- Audio-visual coherence
- Authenticity Index (similaridade entre emoções detectadas em áudio e vídeo)

Saída: **vetor de 16 dimensões**

2.5 Fusion Module

Concatenação:

audio(32) + video(48) + text(32) + cross(16) = 128 dimensões

Esse vetor é o **FeatureVector** final.

3. Cálculo dos Principais Scores

3.1 ImpactScore

(detalhado totalmente na Camada 12)

$\text{dot} = \sum (f_i * w_i) + b$
 $\text{ImpactScore} = 1000 * \text{sigmoid}(\text{dot})$

3.2 Archetype Scores

Vetor de 7 dimensões representando padrões comportamentais detectados pela IA:

- Expressividade
- Criatividade
- Autenticidade Técnica
- Energia Visual
- Clareza Comunicativa
- Coerência Emocional
- Presença Cênica

Cálculo:

```
archetype_scores = softmax(Wa * feature_vector + ba)
```

3.3 RiskScore

Classificador binário:

```
risk_score = sigmoid(Wr * feature_vector + br)
```

Indica risco de:

- sexualidade explícita
- violência
- ódio
- manipulação artificial
- conteúdo impróprio

Definições:

- ≥ 0.70 → auto rejeição
- 0.30–0.69 → revisão humana obrigatória

- $< 0.30 \rightarrow$ conteúdo seguro

3.4 Confidence Score

Define a confiabilidade da inferência:

```
confidence = 1 - (variance(feature_vector))
```

Variância alta = IA está incerta.

4. Pipeline de Execução

Passo 1 — Receber evento `talent.ingested`

Contém:

- paths do vídeo
- metadata
- transcription (se existir)

Passo 2 — Pré-processamento

- normalizar frames
- normalizar espectrogramas
- padronizar sequência textual
- normalizar amplitude do áudio

Passo 3 — Extrair vetores

```
audio_vec = audio_tower(audio)
video_vec = video_tower(frames)
text_vec = text_tower(transcription)
cross_vec = cross_tower(audio, video)
```

Passo 4 — Fusion

```
feature_vector = concat(audio_vec, video_vec, text_vec, cross_vec)
```

Passo 5 — Cálculo dos scores

```
impact_score = calc_impact(feature_vector)
risk_score = calc_risk(feature_vector)
archetypes = calc_archetypes(feature_vector)
confidence = calc_confidence(feature_vector)
```

Passo 6 — Validação

Se `confidence < 0.5` → marcar para moderação humana.

Passo 7 — Persistência

Salvar em `talent_items` :

- `feature_vector`
- `impact_score`
- `risk_score`
- `archetype_scores`
- `confidence`
- `model_hash`

Passo 8 — Emissão de evento

Emitir:

```
talent.inferred
```

para a Camada 4.

5. Logs e Observabilidade

Logs armazenam:

```
{
  "talent_id": "...",
  "latency_ms": 2130,
  "model_hash": "...",
  "impact_score": 742.321,
  "risk_score": 0.22,
  "confidence": 0.87,
  "top_features": ["audio_energy", "expression_happy", "text_clarity"]
}
```

Métricas:

- inference_latency_p95
- drift_monitor
- model_hash_frequency

6. Falhas e Regras de Fallback

6.1 Torre de texto falhou

- Remove texto da fusão
- recalcula pesos
- confidence reduzido

6.2 Torre de vídeo falhou

- IA trabalha só com áudio e texto (degradação leve)

6.3 Torre de áudio falhou

- IA trabalha só com vídeo e texto

6.4 Aurah-Core indisponível

- fallback para `lite-model`

→ scores simplificados

7. Encerramento da Camada 3

A Camada 3 define a **inteligência central** do MST.

Ela transforma conteúdo em matemática — permitindo ranking, moderação, feed e tudo mais.

CAMADA 4 — ORQUESTADOR DE PUBLICAÇÃO (BUSINESS RULES & STATUS PIPELINE)

(Versão técnica completa, detalhada e executável)

1. Propósito da Camada

A Camada 4 controla **todas as decisões de publicação** após a inferência da IA.

Ela é responsável por:

- aplicar regras de negócio,
- validar segurança,
- decidir se o conteúdo será aprovado, limitado, pendente ou rejeitado,
- salvar os dados finais,
- e emitir o evento oficial de publicação.

É a “ponte” entre a análise da IA e a entrada no feed.

2. Entradas da Camada

Recebe o evento:

```
talent.inferred
```

Com os campos:

- `talent_id`
 - `creator_id`
 - `impact_score`
 - `risk_score`
 - `archetype_scores`
 - `feature_vector`
 - `confidence`
 - `metadata`
 - `transcription`
 - `timestamps`
-

3. Saídas da Camada

3.1 Persistência final em Banco

Atualiza:

- `status`
- `visibility`
- `published_at`
- `moderation_flags`
- `recommendation_flags`

3.2 Eventos Kafka

- `talent.published`
- `talent.rejected`
- `talent.limited`
- `talent.manual_review_required`

Esses eventos alimentam feed, moderação, notificações e analytics.

4. Regras de Negócio (Determinísticas)

Essas regras são **matemáticas e objetivas**, não dependem de interpretação humana.

4.1 Regras Baseadas no RiskScore

```
if risk_score >= 0.70 → reject  
if 0.30 <= risk_score < 0.70 → needs_human_review
```

4.2 Regras Baseadas na Confiança da IA

```
if confidence < 0.55 → needs_human_review
```

4.3 Regras de Criadores Novos

Criadores com menos de 7 dias de conta:

```
if impact_score < threshold_low AND account_age < 7d:  
    visibility = limited
```

Threshold padrão:

```
threshold_low = 280
```

4.4 Regras de Conteúdo Repetitivo

Se o criador publicou ≥ 3 conteúdos muito parecidos (similaridade $> 92\%$):

```
visibility = limited  
flag = repetition_detected
```

4.5 Regras de Autenticidade

Se `similaridade do feature_vector com creators verificados  $\geq 0.85$` E

`confidence  $\geq 0.80$` :

```
grant_selo("authenticity")
```

5. Pseudocódigo da Camada (Fluxo Completo)

```
handle_inferred(payload):
```

```
    if payload.risk_score  $\geq 0.70$ :
```

```
        persist(status="rejected", reason="high_risk")
```

```
        emit("talent.rejected")
```

```
        return
```

```
    if  $0.30 \leq$  payload.risk_score  $< 0.70$ :
```

```
        persist(status="needs_human_review")
```

```
        emit("talent.manual_review_required")
```

```
        return
```

```
    if payload.confidence  $< 0.55$ :
```

```
        persist(status="needs_human_review")
```

```
        emit("talent.manual_review_required")
```

```
        return
```

```
    visibility = "public"
```

```
    if is_new_creator(payload.creator_id) and payload.impact_score  $<$  thresh  
old_low:
```

```
        visibility = "limited"
```

```
if is_repetitive_content(payload.talent_id):
    visibility = "limited"

persist(
    status="approved",
    visibility=visibility,
    published_at=now(),
    impact_score=payload.impact_score
)

emit("talent.published")
```

6. Persistência da Camada

Atualiza o registro em `talent_items` :

```
status = approved|rejected|needs_human_review
visibility = public|limited
risk_score = float
impact_score = float
published_at = timestamp
```

7. Logs Estruturados

Exemplo:

```
{
  "talent_id": "uuid",
  "creator_id": "uuid",
  "decision": "approved",
  "risk_score": 0.12,
  "confidence": 0.88,
  "visibility": "public",
```

```
"latency_ms": 73,  
"service": "publication-orchestrator"  
}
```

8. Métricas

- publish_success_rate
- rejection_rate
- manual_review_rate
- avg_publish_latency

Alertas:

- aumento de rejeições > 35%
- queda no publish rate

9. Failover e Consistência

- Transações ACID garantem 1 única publicação
- Eventos possuem `idempotency_key`
- Em caso de falha:
 - reprocessar evento
 - nunca duplicar publicação

10. Integrações Dependentes

A Camada 4 alimenta:

- Feed (Camada 5)
- Engajamento (Camada 6)
- Moderação (Camada 11)
- Eventos & Lives

- Dashboard do criador
 - Ranking e Descoberta
-

11. Encerramento da Camada 4

A Camada 4 é o “juiz técnico” do MST:

decide aprovar, limitar, rejeitar ou encaminhar para revisão humana com base em regras puras, numéricas e consistentes.

Sem ela, o app não garante qualidade e segurança.

CAMADA 5 — INDEXAÇÃO, RANKING E FEED PERSONALIZADO (SEARCH + ES + RANKING FUNCTION)

1. Propósito da Camada

A Camada 5 constrói o **Feed do Mostre Seu Talento**.

Ela é responsável por:

- indexar o conteúdo aprovado,
- armazenar vetores densos para busca semântica,
- calcular o ranking personalizado,
- servir o feed com baixa latência (< 300ms p95),
- e garantir estabilidade, consistência e diversidade de conteúdo.

O feed é onde o usuário **vê os talentos**.

É a vitrine central do MST.

2. Arquitetura Geral da Camada

Envolve 3 sistemas:

2.1 Elasticsearch / OpenSearch

Armazena:

- impact_score
- dense_vector 128d
- metadata
- timestamps
- popularity signals

2.2 Ranking Engine

Calcula FeedScore (pondo peso em impacto, personalização e sinais sociais).

2.3 Cache Layer (Redis)

Armazena:

- pré-listas
 - resultados recentes
 - fallback quando o ES estiver lento/instável
-

3. Documento Indexado no Feed

Formato salvo no ES:

```
{
  "id": "uuid",
  "creator_id": "uuid",
  "published_at": "timestamp",
  "impact_score": 742.321,
  "dense_vector": [...],
  "tags": ["dance","voz"],
  "duration": 36.2,
  "popularity_signals": {
    "waves": 120,
    "donations": 3,
```

```
"views": 2080
},
"archetype_vector": [...],
"thumbnail_url": "https://...",
"language": "pt-BR"
}
```

Campo mais importante:

```
"dense_vector": 128 dimensões
```

Usado para semântica, recomendação e diversidade.

4. Mapeamento no ES

```
"properties": {
  "impact_score": { "type": "float" },
  "published_at": { "type": "date" },
  "dense_vector": {
    "type": "dense_vector",
    "dims": 128,
    "index": true,
    "similarity": "cosine"
  },
},
"tags": { "type": "keyword" },
"popularity_signals": { "type": "object" }
}
```

5. Quando o conteúdo é indexado

Ao receber evento:

```
talent.published
```

O sistema:

1. transforma o payload em documento
2. grava no ES
3. atualiza cache de pré-lista

Latência desejada:

```
index_latency < 100ms
```

6. Fórmula de Ranking (FeedScore)

O feed não usa likes tradicionais.

Ele usa matemática objetiva.

6.1 Fórmula principal

```
FeedScore =  
   $\alpha$  * norm(ImpactScore) +  
   $\beta$  * personalization_score +  
   $\gamma$  * recency_score +  
   $\delta$  * social_signal_score
```

Pesos padrão:

- $\alpha = 0.45$
- $\beta = 0.30$
- $\gamma = 0.20$
- $\delta = 0.05$

6.2 Componentes Explicados

A. norm(ImpactScore)

```
normImpact = ImpactScore / 1000
```

B. personalization_score

Calculado por similaridade com dense_vectors que o usuário já interagiu:

```
personalization_score = cosine(user_vector, talent_vector)
```

C. recency_score

Prioriza conteúdos recentes:

```
recency_score = e^(-λ * hours_since_publish)
```

λ recomendado:

```
λ = 0.04
```

D. social_signal_score

Com base em:

- waves
- views
- donations
- compartilhamentos

Função de suavização:

```
social_signal_score = log(1 + waves + 2*donations)
```

7. Paginação e Continuidade (scroll infinito)

Usa:

```
search_after
```

Garantindo:

- ordenação determinística
 - sem repetição
 - sem buracos
 - baixa latência
-

8. Fallback quando Elasticsearch cai

Se ES estiver indisponível:

8.1 Redis Feed

Servir:

- 50 últimos conteúdos populares
- ordenados por `impact_score`
- TTL = 5 minutos

8.2 Degradação

Sem personalização → feed básico.

9. Logs

Cada busca gera:

```
{  
  "user_id": "uuid",  
  "latency": 87,
```

```
"results_count": 15,  
"cache_hit": false,  
"fallback_used": false}
```

10. Métricas

- feed_latency_p95 < 300ms
- es_index_failure_rate < 0.5%
- cache_hit_rate > 80%
- personalization_success_rate

11. Testes exigidos

- carga (5k buscas/seg)
- rank consistency test
- precisão do dense_vector
- integridade de indexação

12. Encerramento da Camada 5

Essa camada constrói o **coração do feed**.

Ela garante que conteúdo de talentos apareça de forma justa, personalizada, rápida e escalável.

Sem essa camada, o MST não teria como mostrar talentos de forma inteligente.

CAMADA 6 — SISTEMA DE ENGAJAMENTO EM TEMPO REAL (WAVES, VIEWS, NOTIFICAÇÕES)

1. Propósito da Camada

A Camada 6 gerencia **TUDO relacionado ao engajamento** dentro do Mostre Seu Talento:

- registro de visualizações (views)
- ondas de apoio enviadas para o criador (waves)
- contadores
- atualização estatística
- entrega de notificações em tempo real
- integração com ranking e feed
- prevenção de abuso

É uma camada crítica, pois influencia:

- visibilidade do criador
- métricas exibidas no dashboard
- FeedScore
- sistemas sociais
- eventos e convites

2. Eventos de Entrada da Camada

2.1 View

Quando o usuário assiste o conteúdo:

```
POST /mst/{id}/view
```

2.2 Wave

Quando o usuário envia uma onda de apoio:


```
POST /mst/{id}/wave
```

2.3 WebSocket Connection

Para entregar notificações em tempo real.

3. Regras de Contabilização (Matemáticas e Determinísticas)

3.1 Contabilização de Views

Para evitar fraude ou inflar métricas:

A visualização **SÓ conta** quando:

(tempo assistido $\geq$ 2s) OU
(percentual assistido $\geq$ 50%)

Ou seja:

- se a pessoa só passar o dedo rápido → **não conta**
 - se assistir metade do vídeo → **conta**
 - se assistir mais de 2 segundos → **conta**
-

3.2 Contabilização de Waves

Cada wave é um objeto com tipo, exemplo:

- alegria
- força
- incentivo
- gratidão
- coragem

Salvo em:

```
talent_engagements
```

Regra anti-abuso

```
máx 10 waves / minuto / usuário
```

Se ultrapassar:

- começa a ignorar
- aciona flag de abuso se repetido

4. Estrutura da API

4.1 POST /mst/{id}/view

Entrada:

```
{
  "seconds_watched": 5,
  "percentage": 0.38
}
```

Saída:

```
{ "status": "counted" }
```

4.2 POST /mst/{id}/wave

Entrada:

```
{
  "wave_type": "gratitude"
}
```

```
}
```

Saída:

```
{ "status": "ok" }
```

5. Persistência

5.1 Banco

Tabela: `talent_engagements`

id	UUID
talent_id	UUID
user_id	UUID
type	"view" "wave"
wave_type	nullable
amount	nullable
created_at	timestamp

5.2 Contadores agregados

Criados em:

```
talent_items.metrics
```

Exemplo:

```
"metrics": {  
  "views": 2031,  
  "waves": 145,  
  "donations": 2  
}
```

```
}
```

6. Atualização em Tempo Real

6.1 WebSocket

O criador recebe:

```
{  
  "event": "wave_received",  
  "talent_id": "...",  
  "wave_type": "gratitude"  
}
```

6.2 Feed Refresh

Quando novas waves chegam:

- recalcula social_signal_score
- feed pode ser atualizado

7. Fluxo Completo da Camada

1. Recebe requisição de view ou wave
2. Valida usuário
3. Valida rate limit
4. Salva engagement
5. Atualiza contadores
6. Atualiza ES (ranking)
7. Emite evento talent.engagement
8. Entrega WebSocket para o criador

Tudo em menos de:

p95 < 200ms

8. Logs Técnicos

View:

```
{
  "talent_id": "uuid",
  "type": "view",
  "seconds": 4.2,
  "counted": true}
```

Wave:

```
{
  "talent_id": "uuid",
  "type": "wave",
  "wave_type": "gratitude",
  "user_id": "uuid"
}
```

9. Métricas

- view_count_accuracy
- wave_event_latency_p95
- engagement_write_latency_p99
- abuse_detection_rate

Alertas:

- ondas excessivas vindas do mesmo IP
- picos de 10x de waves → possível ataque

10. Failover

10.1 Redis Counter Mode

Se o banco cair:

- waves são contadas em Redis
- flush para banco a cada 30s

10.2 Perda de WebSocket

Notificações pendentes são enviadas via push.

11. Encerramento da Camada 6

A Camada 6 garante que todo o engajamento seja:

- confiável
- rápido
- rastreável
- impossível de fraudar
- integrado com feed e métricas

É a base do impacto social do MST.

CAMADA 7 — SISTEMA DE MICRODOAÇÕES E PAGAMENTOS (FRIENDCOINS + PIX + ESCROW)

1. Propósito da Camada

A Camada 7 gerencia **todo o sistema econômico** do Mostre Seu Talento, incluindo:

- microdoações via FriendCoins
- microdoações via Pix
- controle de escrows
- contabilidade (ledger)
- antifraude
- integração com o dashboard do criador
- taxas do FriendApp
- repasse financeiro

É o coração financeiro da funcionalidade.

2. Arquitetura do Sistema de Pagamentos

A arquitetura utiliza:

2.1 wallet-service

Gerencia FriendCoins:

- saldo
- débito
- crédito
- escrow

2.2 psp-service

Integração com provedores Pix:

- criar cobrança
- receber webhook
- confirmar pagamento

2.3 ledger-service

Sistema contábil interno:

- ACID
- imutável

- auditável

2.4 donation-service

Orquestra doações:

- valida
 - cria transações
 - emite eventos
-

3. Fluxo de Doação — FriendCoins

3.1 Endpoint

```
POST /mst/{id}/donate/fcoins
```

3.2 Entrada

```
{
  "talent_id": "uuid",
  "creator_id": "uuid",
  "amount": 15
}
```

3.3 Validações

- verificar saldo → suficiente?
- verificar limite diário → <= 500 FC/dia
- verificar conta ativa
- verificar se user != criador

3.4 Operações

1. Debitar wallet do usuário
2. Creditar ESCROW do criador
3. Registrar transação no ledger
4. Emitir evento donation.success

3.5 Estrutura do ledger

```
{  
  "id": "uuid",  
  "type": "donation",  
  "currency": "FCOIN",  
  "amount": 15,  
  "from_user": "...",  
  "to_creator": "...",  
  "timestamp": "..."  
}
```

4. Fluxo de Doação — Pix

4.1 Endpoint

POST /mst/{id}/donate/pix

4.2 Processo

1. Criar cobrança no PSP
2. Retornar QR Code para usuário
3. PSP envia webhook → pagamento confirmado
4. Escrow é creditado ao criador
5. Ledger registra transação

6. Evento donation.success emitido

4.3 Campos do webhook

```
{
  "transaction_id": "...",
  "status": "CONFIRMED",
  "amount": 12.50
}
```

5. Escrow do Criador

Todas as doações primeiro entram no **ESCROW** do criador:

Funções:

- evitar chargeback
- proteger contra fraude
- segurar verba até liberação

Tempo de liberação padrão:

D+1 (24 horas)

6. Taxas do FriendApp

Taxa aplicada sobre toda doação:

fee_percentage = 0.05 (5%)

Exemplo:

- Doação = 20

- Taxa FriendApp = 1
 - Líquido criador = 19
-

7. Saque do Criador

Quando o criador quiser sacar:

POST /creator/payout

Validações:

- conta bancária verificada
- KYC aprovado (quando valor mensal > 500)
- sem flags de fraude

Processo:

1. Debitar escrow
2. Transferir via PSP
3. Registrar payout no ledger

8. Antifraude

Regras:

- múltiplas doações entre contas recém-criadas → bloquear
- onda de doações acima de 10x o normal → revisar
- comportamento circular (A doa a B e B doa a A) → alertar
- IPs duplicados suspeitos

Ações:

- congelar escrow
- solicitar verificação

- bloqueio temporário
-

9. Notificações

Quando uma doação acontece:

- criador recebe push
- dashboard atualiza métricas
- feed recalcula social_signal_score

Exemplo WS:

```
{
  "event": "donation_received",
  "amount": 12.50,
  "currency": "PIX"
}
```

10. Logs Técnicos

Doação FriendCoins:

```
{
  "donation_id": "...",
  "type": "fcoin",
  "from": "uuid",
  "to": "uuid",
  "amount": 10,
  "fee": 0.5,
  "status": "success"
}
```

Doação Pix:

```
{
  "transaction_id": "...",
  "amount": 12.50,
  "webhook_verified": true}
```

11. Métricas e Telemetria

- donation_success_rate
- fraud_detection_rate
- escrow_balance_total
- payout_latency_p95
- psp_webhook_delay

Alertas:

- taxa de fraude > 2%
- falhas no PSP
- saques incompletos

12. Failover

12.1 PSP offline

- aceitar doação
- marcar status pending
- processar quando voltar

12.2 Ledger offline

- bloquear novas doações
- emitir alerta crítico

13. Encerramento da Camada 7

A Camada 7 cria o **sistema financeiro completo** do MST.

Ela permite ao usuário apoiar talentos, movimentar FriendCoins, fazer Pix e monetizar o app de modo automatizado, com segurança e integridade contábil.

CAMADA 8 — SISTEMA DE PERFIL DO CRIADOR & MÉTRICAS AVANÇADAS (CREATOR INSIGHTS)

1. Propósito da Camada

A Camada 8 é responsável por construir o **dashboard do criador**, exibindo:

- métricas de impacto
- engajamento
- crescimento
- retenção
- ganhos financeiros
- distribuição de público
- comparativo com conteúdos anteriores
- recomendações técnicas geradas pela IA

É onde o criador entende o próprio progresso e melhora sua performance.

2. Entradas da Camada

A camada consome dados de:

- `talent_items`
- `talent_engagements`

- `ledger`
- `donations`
- `feed_service`
- Aurah-Core (para recomendações)

Todos consolidados por `creator_id`.

3. Estrutura do Dashboard

3.1 Blocos principais:

1. Impacto Geral

- ImpactScore médio
- Conteúdos publicados
- Visibilidade média

2. Engajamento

- Views totais
- Waves recebidas
- Taxa de conclusão (watch-through-rate)
- Engajamento por horário

3. Crescimento

- Novos seguidores
- Comparativo 7 dias / 30 dias

4. Financeiro

- Total de doações
- Total líquido recebido
- Taxas pagas
- Saques realizados

5. Qualidade Técnica

- Presença cênica

- Expressividade
- Coerência áudio-vídeo
- Clareza de comunicação
- Sinais de repetição

6. Recomendações da IA

- pontos fracos
- pontos fortes
- sugestões objetivas

4. API do Dashboard (Creator Insights)

4.1 GET /creator/{id}/insights

Saída:

```
{
  "impact": {
    "average": 682.4,
    "best": 831.2,
    "worst": 431.1
  },
  "engagement": {
    "views": 20431,
    "waves": 541,
    "completion_rate": 0.37
  },
  "growth": {
    "last7": 1021,
    "last30": 4120
  },
  "financial": {
    "donations_total": 341.50,
    "net_received": 309.92,
    "withdrawals": 2
  },
}
```



```
"quality": {  
  "expression": 0.82,  
  "coherence": 0.77,  
  "presence": 0.69  
},  
"recommendations": [...]  
}
```

5. Cálculos Internos das Métricas

5.1 Média de ImpactScore

$$\text{avg_impact} = \Sigma(\text{impact_score}) / \text{total_contents}$$

5.2 Taxa de Conclusão

$$\text{completion_rate} = \text{total_seconds_watched} / \text{total_seconds_available}$$

5.3 Engajamento por Horário

Bucketização em intervalos de 1h:

$$\text{engagement}[\text{hour_bucket}] = \Sigma(\text{waves} + \text{views}) \text{ no período}$$

5.4 Crescimento

$$\begin{aligned}\text{growth_last7} &= \text{followers_now} - \text{followers_7d_ago} \\ \text{growth_last30} &= \text{followers_now} - \text{followers_30d_ago}\end{aligned}$$

5.5 Receitas

```
net_received = gross_donations - fees
```

Fees = 5% (Camada 7)

6. Sistema de Recomendações da IA

Aurah-Core gera 4 tipos de recomendações:

6.1 Técnicas

Ex:

- "Sua iluminação média está baixa. Sugiro gravar em um ambiente mais claro."

6.2 Artísticas

- "Seu vídeo tem excelente presença facial. Mantenha o enquadramento frontal."

6.3 Estratégicas

- "Seu público mais ativo assiste às 20h. Publique nesse horário."

6.4 Conteúdo

- "Seus vídeos musicais geram mais waves do que seus vídeos de humor."
-

7. Pseudocódigo Interno

```
fetch_insights(creator_id):  
  
    items = db.talent_items.where(creator_id)  
  
    impact_avg = avg(items.impact_score)  
    impact_best = max(items.impact_score)
```

```
impact_worst = min(items.impact_score)

views = sum(engagements.views)
waves = sum(engagements.waves)
completion = calc_completion(engagements)

growth = calc_growth(creator_id)

financial = calc_financial(creator_id)

quality = aurah_core.get_quality_vector(creator_id)

recs = aurah_core.generate_recommendations(creator_id)

return all
```

8. Logs Técnicos

```
{
  "creator_id": "...",
  "insights_latency": 42,
  "items_count": 17,
  "recommendations": 4
}
```

9. Métricas

- insights_latency_p95 < 200ms
- recommendation_accuracy_score
- insights_cache_hit_rate

10. Cache

Resultados são cacheados por 1 minuto para reduzir latência.

11. Failover

Se Aurah-Core cair:

```
quality = null  
recommendations = []
```

Dashboard continua funcionando com dados básicos.

12. Encerramento da Camada 8

A Camada 8 transforma dados complexos em INSIGHTS claros e executáveis, permitindo que criadores melhorem sua performance, aumentem sua renda e entendam seu impacto real.

CAMADA 9 — SISTEMA DE DISCOVERY & TALENT MATCHING (RECOMENDAÇÃO AVANÇADA ENTRE PESSOAS)

1. Propósito da Camada

A Camada 9 cria o **sistema que conecta pessoas com talentos compatíveis**, gerando:

- descoberta inteligente
- recomendações de quem assistir
- sugestões de criadores semelhantes

- agrupamento automático de estilos
- expansão do público
- aumento de engajamento
- emparelhamento vibracional (aqui transformado em matemático)

Essa camada é o “descobridor de talentos” do MST.

2. Entradas da Camada

- `feature_vector` dos talentos
 - `archetype_scores` (7 dimensões)
 - `dense_vector` indexado no ES
 - histórico de engajamento do usuário
 - preferências explícitas (categorias)
 - horário de uso do app
 - similaridade social (seguidores em comum)
-

3. Arquitetura do Discovery

3 motores trabalhavam juntos:

3.1 Similarity Engine

Usa:

- Cosine similarity
- ANN (HNSW)
- Clustering automático

3.2 Behavioral Engine

Aprende o gosto do usuário:

- conteúdos vistos
- waves enviadas
- tempo assistido

- rejeições

3.3 Archetype Engine

Correlaciona:

- estilo de performance
 - forma de expressão
 - coerência emo-técnica
-

4. Saídas do Sistema

A camada retorna:

- "pessoas que você deve conhecer"
 - "talentos compatíveis com seu estilo"
 - "talentos que seu público ama"
 - "criadores parecidos com você"
 - "novos talentos para explorar"
-

5. Algoritmos Centrais

5.1 Similaridade Direta

Usa o vetor 128d:

```
similarity = cosine(vecA, vecB)
```

Threshold:

```
similar >= 0.82 → perfis similares
```

5.2 Matching por Afinidade de Estilo

Baseado nos 7 arquétipos:

```
affinity = softmax( dot(archetypeA, archetypeB) )
```

Se afinidade ≥ 0.70 → recomendado.

5.3 Modelo de Preferência Pessoal

Aprende o perfil único do usuário:

```
user_vector = avg( talents_interacted_vectors )
```

Quanto mais waves e visualizações, mais forte a influência.

5.4 Descoberta Diversificada

Para evitar feed repetitivo:

```
diversity_penalty = e^(-λ * similarity_with_last_seen)
```

λ padrão:

```
λ = 0.15
```

5.5 Colaboração Social

Se dois usuários têm padrões parecidos:

```
social_match = cosine(userA_vector, userB_vector)
```

Se `social_match  $\geq 0.80$` → sugerir criadores que o outro usuário segue.

6. Pseudocódigo de Recomendação

```
get_discovery(user):  
  
    user_vec = compute_user_vector(user)  
  
    # 1. Criadores parecidos  
    similar_creators = knn_search(user_vec)  
  
    # 2. Afinidade por estilo  
    style_matches = filter_by_archetype(similar_creators)  
  
    # 3. Diversidade  
    diverse_set = apply_diversity_penalty(style_matches)  
  
    # 4. Ajuste por comportamento  
    personalized = re_rank_by_behavior(diverse_set, user)  
  
    return top_k(personalized)
```

7. API da Camada

GET /mst/discovery

Retorna 20 talentos recomendados.

GET /mst/creator/{id}/similar

Retorna criadores com vetor próximo.

GET /mst/user/{id}/matches

Sugere talentos compatíveis com o gosto do usuário.

8. Cache e Desempenho

Caches usados:

- resultados de similaridade (TTL 15 min)
- clusters pré-computados
- user_vector atualizado a cada 1h

Latência alvo:

```
recommendation_latency_p95 < 180ms
```

9. Logs Técnicos

```
{
  "user_id": "uuid",
  "recommended_count": 20,
  "latency": 127,
  "algorithm_used": "hybrid_v3",
  "diversity_factor": 0.32
}
```

10. Métricas

- recommendation_accuracy
- recommendation_diversity
- discovery_engagement_rate
- bounce_rate

11. Failover e Fallbacks

Se HNSW cluster cair:

- reverter para busca simples por ImpactScore
- sem personalização

- sem similaridade
- apenas “novidades da semana”

Se Dense Vector indisponível:

- usar tags e popularidade (degradação leve)
-

12. Encerramento da Camada 9

A Camada 9 garante que o MST nunca fique repetitivo, ajude criadores a serem descobertos e conecte usuários com talentos compatíveis — tudo por matemática, similaridade de vetores e padrões comportamentais.

CAMADA 10 — SISTEMA DE SELO DE DESTAQUE (HIGHLIGHT BADGES) & VALIDAÇÃO ALGORÍTMICA

1. Propósito da Camada

A Camada 10 cria e gerencia **os selos de destaque** que aparecem no conteúdo dentro do MST.

Esses selos servem para:

- dar reconhecimento técnico
- aumentar visibilidade no feed
- destacar talentos excepcionais
- ajudar usuários e marcas a descobrirem criadores de valor
- incentivar performance e qualidade

E são gerados **automaticamente por algoritmos**, sem favoritismo e sem subjetividade.

2. Tipos de Selos

2.1 Selo “Destaque da Semana”

Conteúdos com performance acima do percentil 95 em:

- ImpactScore
- Waves
- Completion Rate

2.2 Selo “Alta Expressividade”

Baseado em:

```
expression_vector.mean >= 0.75
```

2.3 Selo “Criativo & Original”

Baseado em:

```
textual_uniqueness >= 0.70  
visual_uniqueness >= 0.65
```

2.4 Selo “Performance Técnica”

Vídeo e áudio estáveis, claros e bem produzidos:

```
lighting >= 0.60  
audio_quality >= 0.70  
motion_stability >= 0.65
```

2.5 Selo “Promissor”

Para criadores novos com alta performance:

```
account_age < 30 dias  
AND
```

```
ImpactScore >= 700
```

3. Arquitetura da Camada

A Camada 10 funciona com:

- **badge-engine**: motor de cálculo
- **badge-service**: API
- **badge-publisher**: distribui os selos nos eventos
- **notification-service**: envia notificação ao criador

4. Regras Técnicas de Geração de Selos

4.1 Cálculo diário

Um Job roda diariamente às 04h:

```
cron: 0 4 * * *
```

Ele analisa:

- todos os conteúdos publicados nas últimas 24h
- métricas acumuladas
- scores da IA
- thresholds dinâmicos

4.2 Percentis dinâmicos

Os thresholds são recalculados diariamente:

```
p95_impact = percentile(impact_score, 95)
p95_waves = percentile(waves, 95)
```

```
p95_completion = percentile(completion_rate, 95)
```

Conteúdos acima disso recebem selo “Destaque da Semana”.

4.3 Similaridade e originalidade

Para medir originalidade:

```
visual_uniqueness = 1 - cosine(vec_video, cluster_center)
textual_uniqueness = 1 - cosine(vec_text, cluster_center)
```

Thresholds:

```
visual ≥ 0.65
textual ≥ 0.70
```

4.4 Expressividade

Do vector facial:

```
expression_score = avg(emotion_probs)
if expression_score >= 0.75 → selo
```

4.5 Qualidade técnica

Com base em:

- iluminação
- clareza de áudio
- estabilidade
- foco

Fórmula:

```
tech_quality = (lighting*0.3 + audio_quality*0.4 + stability*0.3)
```

Threshold:

```
tech_quality >= 0.70
```

5. Persistência no Banco

Tabela `talent_badges`:

```
id          UUID
talent_id   UUID
badge_type  VARCHAR
score       FLOAT
generated_at TIMESTAMP
```

Index:

```
idx_talent_badge_talent_id
```

6. Eventos Emitidos

Quando um selo é concedido:

```
talent.badge.granted
```

Payload:

```
{
  "talent_id": "uuid",
```

```
"badge": "highlight_week",  
"score": 842.1  
}
```

7. Integrações da Camada

Essa camada alimenta:

- **Feed (Camada 5)** → conteúdos com selo recebem boost leve
- **Discovery (Camada 9)** → priorização de criadores
- **Eventos (Camada 17+)** → convites para competições
- **Dashboard do Criador (Camada 8)** → exibe selos recebidos

8. Boost no Ranking

Conteúdos com selo recebem aumento exclusivo no ranking:

$$\text{FeedScore} = \text{FeedScore} * (1 + \text{badge_boost})$$

Valores:

```
highlight_week = +12%  
expressiveness = +5%  
creative = +4%  
tech = +3%  
promissor = +8%
```

9. Notificações

Via WebSocket e push:

"Parabéns! Seu talento recebeu o selo: Destaque da Semana"

10. Segurança e Anti-Abuso

- nenhum moderador concede selo manualmente
- IA não pode ser forçada
- regras são matemáticas
- thresholds diários evitam manipulação

Logs:

```
threshold_version  
badge_engine_hash
```

11. Pseudocódigo Completo

for each talent published in last 24h:

```
if impact_score >= p95_impact AND  
  waves >= p95_waves AND  
  completion_rate >= p95_completion:  
  grant("highlight_week")
```

```
if expression_score >= 0.75:  
  grant("expressiveness")
```

```
if visual_uniqueness >= 0.65 AND textual_uniqueness >= 0.70:  
  grant("creative")
```

```
if tech_quality >= 0.70:  
  grant("tech")
```



```
if account_age < 30d AND impact_score >= 700:  
    grant("promissor")
```

12. Encerramento da Camada 10

A Camada 10 cria um sistema **justo, automatizado e matemático** de reconhecimento.

Ela valoriza criadores, aumenta a entrega do app e mantém a meritocracia técnica sem influência humana.

CAMADA 11 — PIPELINE DE MODERAÇÃO AUTOMÁTICA + FILA HUMANA (RISK ENGINE)

1. Propósito da Camada

A Camada 11 é responsável por **garantir a segurança do conteúdo** e proteger:

- usuários
- criadores
- menores de idade
- a reputação do FriendApp
- os parceiros

Ela combina:

- IA automática
- classificadores multimodais
- heurísticas determinísticas
- revisão humana (quando necessária)

- logs completos
- rastreabilidade

E decide com 100% de objetividade:

aprovar, limitar, rejeitar ou enviar para revisão humana.

2. Entradas da Camada

A Camada 11 recebe o evento:

```
talent.inferred
```

Com os atributos:

- risk_score
 - confidence
 - archetype_scores
 - transcription
 - metadata (face detection, brightness, áudio)
 - histórico do criador
 - linguagem detectada
-

3. Componentes Internos do Sistema

3.1 risk-engine

Classificador técnico baseado no vetor 128d.

3.2 rule-engine

Heurísticas mecânicas e matemáticas.

3.3 moderation-queue

Fila para moderadores humanos.

3.4 audit-log

Rastreia decisões tomadas.

4. Classificações Possíveis

A Camada 11 define 4 possíveis saídas:

4.1 APPROVED

Conteúdo seguro.

4.2 LIMITED

Visibilidade reduzida automaticamente.

4.3 REJECTED

Conteúdo proibido ou ilegal.

4.4 HUMAN_REVIEW

Moderador precisa avaliar.

5. Regras Automáticas (Determinísticas)

Essas regras NÃO dependem de opinião humana.

5.1 Baseadas no RiskScore

```
if risk_score >= 0.70 → REJECTED
```

```
if 0.30 <= risk_score < 0.70 → HUMAN_REVIEW
```

5.2 Baseadas no Confidence Score

```
if confidence < 0.55 → HUMAN_REVIEW
```

5.3 Baseadas no Conteúdo do Texto

Busca padrões proibidos:

Palavras-chave proibidas

(sempre REJECT):

- abuso sexual
- pornografia explícita
- discurso de ódio explícito
- incitação à violência

Palavras sensíveis

(envia para review):

- automutilação
- depressão severa
- violência leve
- bullying

5.4 Baseadas em Heurísticas Visuais

Usando dados da Camada 2:

Face Oculta + áudio sexualizado

→ HUMAN_REVIEW

Brilho muito baixo

```
if brightness < 0.18 → HUMAN_REVIEW
```

Movimento suspeito

```
if motion_level > 0.85 → HUMAN_REVIEW
```

5.5 Regras baseadas no criador

Criadores marcados com flags:

```
if creator under observation → HUMAN_REVIEW
```

Criadores novos (menos de 7 dias):

```
risk_score_threshold = 0.20
```

6. Pipeline Completo de Moderação

1. Recebe talent.inferred
2. Aplica risk rules
3. Aplica confidence rules
4. Aplica heurísticas (texto, vídeo, áudio)
5. Classifica:
 - APPROVED
 - LIMITED
 - REJECTED
 - HUMAN_REVIEW
6. Persiste status
7. Emite evento correspondente
8. (Se for HUMAN_REVIEW) adiciona à fila de moderação

7. Persistência

Campos atualizados na tabela `talent_items`:

```
risk_score
confidence
status = approved|limited|rejected|review
moderation_flags[]
moderation_reason
review_required
```

8. Fila de Revisão Humana

Quando HUMAN_REVIEW:

- conteúdo entra na fila
- moderador tem SLA de 10 min
- decisões são registradas
- moderador envia:
 - APPROVE
 - LIMIT
 - REJECT

Persistência:

```
moderation_reviews
```

Campos:

```
reviewer_id
talent_id
decision
reason
timestamp
```

9. Eventos Emitidos

9.1 Se approved

```
talent.moderation.approved
```

9.2 Se rejected

```
talent.moderation.rejected
```

9.3 Se limited

```
talent.moderation.limited
```

9.4 Se human review

```
talent.moderation.queue
```

10. Segurança e Auditoria

Tudo é auditado:

```
{
  "talent_id": "...",
  "risk_score": 0.82,
  "decision": "REJECTED",
  "rule_triggered": "risk_score >= 0.70",
  "timestamp": "..."
}
```

Logs são imutáveis.

11. Métricas

- false_positive_rate
- false_negative_rate
- human_review_load
- moderation_latency_p95
- rejection_rate
- limited_rate

Alertas:

- aumento súbito de rejeições
 - moderadores lentos
 - falhas nas regras automáticas
-

12. Failover

Se risk-engine cair:

```
status = HUMAN_REVIEW
```

Se fila humana ficar cheia:

```
status = LIMITED
```

13. Encerramento da Camada 11

A Camada 11 garante que o Mostre Seu Talento permaneça **seguro, legal, confiável, auditável e justo**, combinando IA + regras matemáticas + revisão humana.

É uma das camadas mais críticas do sistema.

CAMADA 12 — SISTEMA DE PONTUAÇÃO DE IMPACTO (IMPACTSCORE) — MODELAGEM MATEMÁTICA COMPLETA

1. Propósito da Camada

A Camada 12 define **a fórmula oficial** que decide a relevância do conteúdo dentro do MST:

- Prioridade no feed
- Descoberta
- Boosts
- Exposição
- Convites para eventos
- Destaques
- Relevância temporal

É a métrica mais importante de todo o sistema.

2. Entradas do ImpactScore

O cálculo usa:

2.1 Feature Vector (128 dimensões)

Fornecido pela Camada 3:

- áudio (32d)
- vídeo (48d)

- texto (32d)
- cross-modal (16d)

2.2 Vetor de Pesos (128 dimensões)

Treinado por regressão L2 regularizada.

2.3 Bias (1 valor)

2.4 Normalização das features

Todas as features são padronizadas:

```
mean = 0  
std = 1
```

3. Fórmula Completa do ImpactScore

3.1 Produto interno

```
dot =  $\sum (f_i \times w_i) + b$ 
```

3.2 Função de ativação

Usa-se **sigmoid** para garantir que o valor fique entre 0 e 1:

```
 $\sigma(x) = 1 / (1 + e^{-x})$ 
```

3.3 Escala final

```
ImpactScore = 1000  $\times$   $\sigma(\text{dot})$ 
```

4. Interpretação da Escala

- 0–200 → baixo impacto
 - 200–400 → médio-baixo
 - 400–600 → mediano
 - 600–800 → alto
 - 800–1000 → excepcional
-

5. Estrutura do Modelo

5.1 Vetor de features

128 floats normalizados.

5.2 Pesos

Armazenados como:

```
weights.bin  
bias.bin  
model_hash
```

5.3 Tamanho

128 valores para w + 1 para b.

6. Categorização das Features

6.1 Áudio

- energy_mean
- energy_std
- mfcc_01...mfcc_13
- pitch_var

- prosody_vector (8d)

6.2 Vídeo

- expression probs (5d)
- face presence
- lighting score
- motion magnitude
- stability score
- frame_embeddings (redução PCA → 32d)

6.3 Texto

- sentiment_score
- emotion_probs (5d)
- readability
- clarity_score
- embedding reduced (8d)

6.4 Cross-modal

- audio-video alignment
- coherence index
- authenticity index

7. Pipeline de Cálculo

1. receive feature_vector
2. normalize features
3. $\text{dot} = \sum(f_i \times w_i) + b$
4. $\text{activated} = \text{sigmoid}(\text{dot})$
5. $\text{ImpactScore} = \text{activated} \times 1000$
6. save to DB

8. Persistência

Campo no banco:

```
impact_score NUMERIC(6,3)
```

Valores como:

```
742.321  
568.910
```

9. Fallbacks

9.1 Falha no modelo principal

Usar modelo leve:

```
ImpactScore = 10 × (audio_score*0.4 + video_score*0.4 + text_score*0.2)
```

9.2 Sem texto (falha no ASR)

Recalcular pesos só com áudio + vídeo.

9.3 Vetor incompleto

Remover dimensões faltantes e renormalizar.

10. Logs Técnicos

```
{  
  "talent_id": "...",  
  "impact": 742.321,  
  "dot": 0.91,  
  "confidence": 0.88,
```

```
"top_features": ["pitch_var","lighting","coherence"],  
"model_hash": "afe293..."  
}
```

11. Métricas de Qualidade do Modelo

- MAE
- RMSE
- drift score
- distribuição média
- estabilidade temporal

Monitorado por:

```
impactscore_drift_detector
```

12. Testes Essenciais

12.1 Teste de Reprodutibilidade

Mesmo input → mesmo output.

12.2 Teste de Ruído

Pequenas mudanças no input:

```
variação < 3%
```

12.3 Teste de Deriva

KS-test semanal.

12.4 Teste de limite

- feature_vector zeros
 - valores extremos
-

13. Integrações Dependentes

O ImpactScore alimenta:

- Feed Ranking (Camada 5)
 - Discovery (Camada 9)
 - Selos (Camada 10)
 - Eventos e Lives
 - Dashboard do Criador (Camada 8)
 - Análise de crescimento
-

14. Encerramento da Camada 12

A Camada 12 define o **marco matemático central** do MST.

Ela converte a performance do criador em um número objetivo, justo, auditável e completamente automatizado.

Sem ela, nada pode ser ranqueado ou recomendado.

CAMADA 13 — SISTEMA DE ARMAZENAMENTO E ESTRUTURA DE DADOS (DATA LAYER & SCHEMA GLOBAL)

1. Propósito da Camada

A Camada 13 define **toda a infraestrutura de dados** do Mostre Seu Talento.

Ela especifica:

- bancos

- tabelas
- formatos
- índices
- relacionamentos
- versionamento
- regras de consistência
- estrutura de armazenamento multimídia

É o alicerce que garante que tudo funcione, seja rápido, escalável e organizado.

2. Arquitetura Completa do Data Layer

O MST usa **3 sistemas principais**:

2.1 PostgreSQL (Banco Relacional Principal)

Armazena:

- talentos
 - métricas
 - engajamentos
 - transcrições
 - decisões de moderação
 - badges
 - relacionamentos
-

2.2 S3-Compatible Storage (Armazenamento de Arquivos)

Armazena:

- vídeo
- áudio

- thumbnails
 - frames
 - JSON de transcrição
 - metadados brutos
-

2.3 Elasticsearch / OpenSearch (Busca + Ranking)

Armazena:

- dense_vector (128d)
 - ImpactScore
 - signals sociais
 - tags
 - preview
-

3. Estrutura de Pastas no Storage (S3)

Padrão:

```
s3://friendapp/mst/{creator_id}/{talent_id}/
├── original.mp4
├── processed_720p.mp4
├── audio.wav
├── transcription.json
├── frames/
│   ├── frame_001.jpg
│   └── frame_002.jpg
├── thumbnails/
│   ├── thumb_small.jpg
│   ├── thumb_medium.jpg
│   └── thumb_large.jpg
└── metadata.json
```

Regras:

- TTL de 365 dias para frames
 - thumbnails sempre preservadas
 - conteúdo antigo pode ir para glacier-storage
-

4. Esquema do Banco (PostgreSQL)

4.1 Tabela: talent_items

```
CREATE TABLE talent_items (  
  id UUID PRIMARY KEY,  
  creator_id UUID REFERENCES users(id),  
  title VARCHAR(200),  
  description TEXT,  
  intention VARCHAR(250),  
  media_refs JSONB,  
  impact_score NUMERIC(6,3),  
  archetype_scores JSONB,  
  risk_score NUMERIC(3,2),  
  confidence NUMERIC(3,2),  
  feature_vector BYTEA,  
  model_hash VARCHAR(100),  
  status VARCHAR(30),  
  visibility VARCHAR(30),  
  metadata JSONB,  
  metrics JSONB DEFAULT '{}',  
  created_at TIMESTAMPTZ DEFAULT now(),  
  published_at TIMESTAMPTZ,  
  updated_at TIMESTAMPTZ DEFAULT now()  
);
```

5. Índices Essenciais

```
CREATE INDEX idx_talent_creator ON talent_items(creator_id);  
CREATE INDEX idx_talent_published ON talent_items(published_at DESC);
```

```
CREATE INDEX idx_talent_status ON talent_items(status);
CREATE INDEX idx_talent_visibility ON talent_items(visibility);
CREATE INDEX idx_talent_tags ON talent_items USING GIN ((metadata→'tags'));
```

6. Tabela: talent_engagements

```
CREATE TABLE talent_engagements (
  id UUID PRIMARY KEY,
  talent_id UUID REFERENCES talent_items(id),
  user_id UUID REFERENCES users(id),
  type VARCHAR(20),      -- view, wave, donation
  wave_type VARCHAR(20),
  amount NUMERIC(12,2),
  seconds_watched NUMERIC(6,2),
  created_at TIMESTAMPTZ DEFAULT now()
);
```

Índice:

```
CREATE INDEX idx_engagement_talent ON talent_engagements(talent_id);
```

7. Tabela: moderation_reviews

```
CREATE TABLE moderation_reviews (
  id UUID PRIMARY KEY,
  talent_id UUID REFERENCES talent_items(id),
  reviewer_id UUID REFERENCES moderators(id),
  decision VARCHAR(20),
  reason TEXT,
  timestamp TIMESTAMPTZ DEFAULT now()
```

```
);
```

8. Tabela: talent_badges

```
CREATE TABLE talent_badges (  
  id UUID PRIMARY KEY,  
  talent_id UUID REFERENCES talent_items(id),  
  badge_type VARCHAR(50),  
  score FLOAT,  
  generated_at TIMESTAMPTZ DEFAULT now()  
);
```

9. Índices Globais do Banco

- **btree** para ordenação
- **GIN** para JSON
- **Hash** para status

10. Estrutura no ElasticSearch

Documento indexado:

```
{  
  "id": "uuid",  
  "creator_id": "uuid",  
  "impact_score": 742.321,  
  "published_at": "2025-01-21T12:00:00Z",  
  "dense_vector": [... 128 floats ...],  
  "tags": ["dance", "voz"],  
  "popularity_signals": {  
    "waves": 130,  
    "views": 2084,  
  }  
}
```

```
"donations": 4
},
"archetype_vector": [...],
"thumbnail_url": "https://...",
"duration": 34.2
}
```

11. Mapeamento do Dense Vector

```
"dense_vector": {
  "type": "dense_vector",
  "dims": 128,
  "similarity": "cosine",
  "index": true}
```

12. Regras de Versionamento

12.1 Conteúdo

- cada edição incrementa `version`
- histórico salvo em coleção de auditoria

12.2 Vetores

Cada inferência salva:

- `feature_vector`
- `model_hash`
- `explanation`

12.3 Métricas

Snapshots semanais.

13. Regras de Integridade

- `creator_id` deve existir em `users`
 - `impact_score` nunca pode ser NULL
 - `status` $\in$ {pending, approved, rejected, limited, review}
 - `media_refs` sempre contém pelo menos o processed video
-

14. Backup e Disaster Recovery

Banco

- backups diários
- retenção: 30 dias
- PITR (Point-in-time recovery) ativo

S3

- versionamento ativado
- replicação cruzada (opcional)

ElasticSearch

- snapshot automático
 - recriação via replay de Kafka se necessário
-

15. Observabilidade

Métricas:

- `db_query_latency_p95`
 - `es_index_latency`
 - `s3_upload_latency`
 - `ingestion_failure_rate`
-

16. Encerramento da Camada 13

A Camada 13 define **toda a fundação de dados do MST**, garantindo:

- consistência
- velocidade
- escalabilidade
- rastreabilidade
- segurança
- estrutura limpa para desenvolvedores

Toda camada posterior depende desta.

CAMADA 14 — SISTEMA DE INTERFACE: FLUXO DO USUÁRIO (UI/UX FLOW + COMPONENTES VISUAIS + EVENTOS)

1. Propósito da Camada

A Camada 14 define **TODA a experiência visual e interativa** que o usuário tem dentro do Mostre Seu Talento.

Ela estabelece:

- estrutura das telas
- componentes reutilizáveis
- animações
- eventos de UI
- regras de navegação
- validação de inputs

- padrões de interação
- fallback para conexão fraca
- acesso para usuários com deficiência

É o manual técnico que o time de front-end usa para construir a interface.

2. Arquitetura de Interface (Front-End)

Projeto baseado em:

- **React Native** (Android/iOS)
- **Ciclo unidirecional** (Redux / Zustand)
- **Lazy loading** para vídeos
- **Skeletons** para carregamento
- **WebSockets** para notificações

Padrões utilizados:

- Atomic Design
 - Componentização total
 - Responsividade
 - Suporte a tema escuro
-

3. Telas Principais do MST

3.1 Tela 1 — Feed de Talentos

Componentes:

- VideoCard
- Player inline
- Botão de Waves
- Botão de Compartilhar
- Loading progress
- BadgeViewer

Ações:

- swipe para próximo
 - tap para pausar/retomar
 - press para detalhes
 - double tap → envia wave rápida
-

3.2 Tela 2 — Criar Talento (Gravação/Upload)

Componentes:

- CameraPreview
- RecordButton
- UploadButton
- ProgressBar
- Timer
- PermissionBanner
- ErrorToast

Regras:

- máximo 60s
 - mínimo 3s
 - validação automática de microfone e câmera
-

3.3 Tela 3 — Edição e Publicação

Componentes:

- ThumbnailSelector
- InputTitle
- InputDescription
- InputIntention
- LanguageSelector
- TagGenerator

Ações:

- cortar vídeo
 - revisar thumbnail
 - editar transcrição (opcional)
-

3.4 Tela 4 — Detalhes do Talent

Componentes:

- PlayerFull
 - MetricsBar
 - CommentsSection (futuro)
 - WavesList
 - SuggestedCreators (Camada 9)
 - DonateButton
-

3.5 Tela 5 — Perfil do Criador

Componentes:

- CreatorHeader
 - StatsOverview
 - BadgesCarousel
 - GridVideos
 - InsightButton
-

3.6 Tela 6 — Insights (Camada 8)

Componentes:

- ChartLine
 - ChartBar
 - RecommendationBanner
 - GrowthWidget
-

4. Máquina de Estados da Interface

A UI segue uma máquina de estados para evitar erros:

IDLE → LOADING → READY → ERROR → RETRY

4.1 Estados detalhados:

IDLE

Nada foi carregado.

LOADING

Skeletons exibidos.

READY

Conteúdo disponível.

ERROR

Exibe:

"Tente novamente"

RETRY

Reexecuta chamada de API.

5. Interações do Player (Vídeo)

5.1 Gestos:

- Tap → pausar/retomar
- Double tap → enviar wave
- Swipe vertical → navegar
- Long press → abrir opções avançadas

5.2 Estados internos:

BUFFERING → PLAYING → PAUSED → ENDED → ERROR

5.3 Eventos disparados:

- `onPlay`
 - `onPause`
 - `onWatch(seconds)`
 - `onEnd`
 - `onBufferStart`
 - `onBufferEnd`
 - `onWaveSend`
-

6. Sistema de Animações

6.1 Animação de Waves

Quando usuário envia uma wave:

- animação sobe na tela
- cor depende da wave
- desaparece em 0.6s

6.2 Animação de Carregamento

- skeletons
- shimmer effect

6.3 Animações de Navegação

- transições suaves (ease-in-out)
 - fade + slide
-

7. Componentes Reutilizáveis (Atomic Design)

7.1 Átomos

- Botões
- Ícones
- Avatares
- TextInputs
- Chips
- Loaders

7.2 Moléculas

- VideoCard
- MetricsBar
- WavesButton
- DonateButton
- BadgeViewer

7.3 Organismos

- FeedList
 - CreatorProfile
 - TalentDetail
 - InsightsScreen
-

8. Chamadas de API Integradas ao Front-End

GET /mst/feed

Retorna vídeos para o feed.

POST /mst/{id}/view

Enviado quando o usuário assiste.

POST /mst/{id}/wave

Envia wave.

POST /mst/create

Cria conteúdo.

POST /mst/commit

Finaliza criação.

GET /creator/{id}/insights

Tela de métricas.

9. Sistema de Erros e Mensagens

9.1 Erros de rede

"Sem conexão no momento"

9.2 Erros de API

"Algo deu errado, tente novamente"

9.3 Erros críticos

"O vídeo não pôde ser processado"

10. Acessibilidade

- suporte a leitores de tela
 - contraste ajustado
 - toques maiores
 - animações desativáveis (modo reduzido)
-

11. Caching

- vídeos via caching local
 - thumbnails pré-baixadas
 - preferência do usuário salva localmente
-

12. Segurança no Front-End

- tokens armazenados com secure storage
 - proteção contra replay
 - desabilitar gravação em contas bloqueadas
-

13. Encerramento da Camada 14

A Camada 14 define toda a estrutura visual e lógica de interação do MST.

É o “manual oficial de front-end”, garantindo consistência, fluidez e experiência impecável para o usuário.

CAMADA 15 — SISTEMA DE NOTIFICAÇÕES EM TEMPO REAL (WEBSOCKETS + PUSH + EVENTOS)

1. Propósito da Camada

A Camada 15 gerencia **todas as notificações** relacionadas ao Mostre Seu Talento, incluindo:

- notificações em tempo real (WebSocket)
- push notifications (Firebase/APNs)
- mensagens internas
- alertas de moderação
- confirmações de doações
- eventos especiais
- selos recebidos
- convites

O objetivo é manter o usuário conectado, informado, motivado e engajado.

2. Arquitetura da Camada

2.1 notification-service

Microserviço principal.

2.2 WebSocket Gateway

Para mensagens instantâneas.

2.3 Push Provider

- Firebase (Android)
- APNs (iOS)

2.4 Queue

Fila Kafka:

```
notifications.queue
```

2.5 Templates

Arquivos JSON com padrões de mensagens.

3. Tipos de Notificações

3.1 Em tempo real (WebSocket)

- recebeu wave
- recebeu doação
- comentário (futuro)
- selo recebido
- criação aprovada

3.2 Push Notification

- momento de pico do público
 - talent publicado
 - insights semanais
 - convites de eventos
 - destaques da semana
-

4. Eventos que geram notificações

Evento	Notificação
<code>talent.published</code>	"Seu novo talento está no ar!"
<code>talent.engagement</code>	"Você recebeu uma wave!"
<code>donation.success</code>	"Você recebeu uma doação!"
<code>talent.badge.granted</code>	"Você ganhou um novo selo!"
<code>moderation.limited</code>	"Seu conteúdo teve visibilidade reduzida."
<code>moderation.rejected</code>	"Seu conteúdo foi rejeitado."

5. Payloads de Notificação

5.1 Real-time WebSocket

```
{
  "event": "wave_received",
  "talent_id": "uuid",
  "wave_type": "gratitude",
  "from_user": "uuid",
  "timestamp": "...
}
```

5.2 Push Notification

```
{
  "title": "Parabéns!",
  "body": "Seu conteúdo recebeu um novo selo.",
  "data": {
    "talent_id": "uuid",
    "badge": "highlight_week"
  }
}
```

6. Regras de Entrega

6.1 WebSocket aberto

Entrega instantânea.

6.2 WebSocket fechado

Caixa de mensagens offline, depois push.

6.3 Push em horários inteligentes

Notificações só chegam no horário de maior atividade do usuário:

```
user_activity_peak_hours = modelo_IA
```

Baseado em:

- uso do app
- engajamento
- histórico de horários

7. Anti-Spam

Máx por hora:

- 6 push notifications
- 10 WebSocket non-critical

Agrupamento automático:

"Você recebeu 12 waves"

8. Pseudocódigo da Camada

onEvent(e):

template = select_template(e.type)

payload = render(template, e)

if user.online:

 websocket.send(payload)

else:

 push.send(payload)

9. Persistência

Tabela: `notifications_log`

```
id UUID
user_id UUID
type VARCHAR(30)
payload JSONB
delivered BOOLEAN
timestamp TIMESTAMPTZ
```

10. Monitoramento e Métricas

- `delivery_rate`
- `websocket_latency_p95`
- `push_conversion_rate`
- `offline_queue_size`
- `notification_spam_rate`

Alertas:

- WebSocket desconectando demais
- Fila de notificações crescendo
- Push com baixa conversão

11. Failover

Se WebSocket falhar:

```
fallback → push immediately
```

Se push falhar:

- retentativa: 10s, 30s, 2min, 5min

12. Encerramento da Camada 15

Essa camada garante que o MST seja **vivo**, dinâmico e conectado.

Sem a Camada 15:

- ninguém saberia quando recebe wave
- nem doação
- nem selo
- nem aprovação
- nem convites
- nem insights

Ela mantém o coração social pulsando.

CAMADA 16 — SISTEMA DE DENÚNCIAS, SEGURANÇA COMUNITÁRIA & ANTIFRAUDE SOCIAL

1. Propósito da Camada

A Camada 16 garante que o MST tenha:

- mecanismo de denúncia seguro
- prevenção contra abuso
- limitação de contas mal-intencionadas
- inteligência para detectar comportamentos nocivos
- proteção a menores
- ambiente saudável e não tóxico

Ela funciona EM PARALELO com a Camada 11 (moderação), mas focada em **interações humanas**, não no conteúdo em si.

2. Elementos do Sistema

O sistema é composto por:

- **report-service** (API de denúncias)
 - **community-risk-engine** (modelo de comportamento)
 - **fraud-behavior-detector**
 - **moderation-dashboard**
 - **rate-limit-service**
 - **audit-log**
-

3. Tipos de Denúncia

Usuários podem denunciar por:

1. Nudez / Conteúdo Sexual
 2. Violência
 3. Discurso de ódio
 4. Importunação sexual
 5. Menores em risco
 6. Golpe / fraude
 7. Spam
 8. Comportamento abusivo
 9. Conteúdo enganoso
 10. Identidade falsa
-

4. API de Denúncia

POST /mst/{id}/report

Entrada:

```
{
  "reason": "harassment",
  "details": "usuário fez comentários ofensivos"
}
```

Resposta:

```
{ "status": "received" }
```

Validações:

- 1 denúncia por conteúdo por usuário
- detalhes ≤ 500 caracteres
- usuário autenticado

5. Modelo de Risco Comportamental

O sistema gera um `community_risk_score` baseado em:

5.1 Sinais negativos

- denúncias recebidas
- denúncias confirmadas
- picos de denúncias em múltiplos conteúdos
- uso de expressões agressivas (texto)
- waves mal-intencionadas repetidas
- histórico de tentativas de fraude financeira
- rejeições por moderação (Camada 11)

5.2 Sinais positivos

- tempo de conta
- conteúdos aprovados

- interações positivas
 - ausência de denúncias recentes
-

5.3 Fórmula

```
community_risk_score = sigmoid(  
    w1*reports +  
    w2*confirmed_reports +  
    w3*toxicity +  
    w4*fraud_attempts -  
    w5*positive_history  
)
```

Escala:

- ≥ 0.75 → CONTA SOB RESTRIÇÃO
 - $0.45-0.74$ → MONITORAMENTO
 - < 0.45 → NORMAL
-

6. Ações Automáticas

Dependendo do risco e denúncias:

6.1 Conteúdo removido automaticamente

Se relatório confirmado por IA + outros usuários.

6.2 Limitação de conta

Usuário só pode postar 1 conteúdo a cada 24h.

6.3 Shadow Ban

Conteúdo fica invisível, sem alertar o usuário (proteção comunitária).

6.4 Notificação educativa

Mensagem enviada:

"Mantenha o respeito. Seu comportamento foi sinalizado."

6.5 Bloqueio total

Se risco ≥ 0.85 .

7. Pseudocódigo Geral

```
onReport(talent_id, reporter_id, reason):  
  
    save_report()  
  
    score = risk_engine.evaluate(talent_id)  
  
    if score >= 0.80:  
        auto_remove(talent_id)  
        notify_creator()  
        notify_moderation_team()  
  
    else if score >= 0.60:  
        queue_review(talent_id)  
  
    update_user_risk(creator_id)
```

8. Detecção de Abuse Patterns

8.1 Report Bombing

Múltiplas denúncias coordenadas.

→ O sistema ignora denúncias duplicadas e usa peso reduzido.

8.2 Cyberstalking

Usuário denunciando a mesma pessoa repetidamente.

→ Bloqueia denúncias por 7 dias.

8.3 Spam

Detectado por:

- repetição de mensagens
- URLs suspeitas
- comportamento robótico

Ação:

shadow-ban automático

9. Painel da Moderação

Exibe:

- denúncias recentes
 - nível de risco dos criadores
 - conteúdo pendentes
 - medidas automáticas tomadas
 - histórico completo de logs
-

10. Logs Técnicos

```
{
  "talent_id": "...",
  "reporter_id": "...",
  "reason": "harassment",
  "risk_before": 0.45,
  "risk_after": 0.61,
  "action": "queued_review"
```

```
}
```

11. Métricas da Camada

- abuse_detection_rate
- false_report_rate
- moderation_latency
- community_risk_avg
- auto_removal_accuracy

Alertas:

- surto de denúncias
- falsas denúncias > 20%
- risco médio subindo

12. Failover

Se o report-service cair:

- salvar denúncia localmente
- enviar em batch depois

Se risk-engine cair:

- só moderação humana decide

13. Encerramento da Camada 16

A Camada 16 garante que o MST seja **seguro, acolhedor e protegido**, tanto por IA quanto pela própria comunidade.

Ela evita abuso, fraude, assédio e agressões.

É um dos pilares sociais do sistema.

CAMADA 17 — SISTEMA DE CONVITES, EVENTOS E PALCOS (INTEGRAÇÃO MST ↔ EXPERIÊNCIAS REAIS)

1. Propósito da Camada

Esta camada conecta o Mostre Seu Talento com:

- eventos internos do FriendApp
- oportunidades reais em bares, casas de show, restaurantes, festivais
- palcos vibracionais (lives, batalhas, festivais internos)
- convites diretos para criadores de destaque

Aqui nasce a ponte entre talento → reconhecimento → oportunidade real.

2. Elementos do Sistema

2.1 event-service

Gerencia todos os eventos.

2.2 invite-engine

Decide quem deve ser convidado com base nos dados MST.

2.3 talent-selector

Filtro matemático selecionando criadores compatíveis.

2.4 partner-gateway

Canal para empresas enviarem convites para talentos.

2.5 webhook de oportunidades

Retorno quando um criador aceita.

3. Tipos de Eventos Integrados ao MST

1. Palco MST (interno do app)

- apresentações curtas
- lives
- batalhas saudáveis
- destaques da semana

2. Eventos presenciais externos

- bar contratando cantor
- restaurante buscando música ao vivo
- casa noturna procurando DJ
- hotel buscando animador

3. Eventos híbridos

- transmissões ao vivo + presença física
- festivais FriendApp

4. Eventos automáticos da IA

- "Semana das Vozes"
- "Festival Criadores"
- "Maratona MST"

4. Critérios Matemáticos Para Convites

Convites NÃO são manuais.

O sistema usa cálculos objetivos baseados em:

4.1 Score de Oportunidade

```
opportunity_score =  
(impact_score * 0.45) +  
(expressiveness * 0.20) +  
(tech_quality * 0.15) +
```

```
(audience_growth * 0.10) +  
(completion_rate * 0.10)
```

4.2 Compatibilidade com evento

Cada evento possui:

- categoria
- estilo
- faixa etária
- requisitos técnicos

Criador compatível se:

```
similarity(event_vector, creator_vector) >= 0.75
```

4.3 Critérios de segurança

Criador só é convidado se:

```
community_risk_score < 0.45  
status ≠ review  
status ≠ limited
```

5. Fluxo Completo de Geração de Convite

1. partner cria evento
2. app gera event_vector
3. engine consulta MST
4. filtra talentos compatíveis
5. calcula opportunity_score
6. ranqueia pela pontuação

7. seleciona top 10
8. envia convites
9. criador aceita ou recusa

6. API — Eventos e Convites

6.1 Criar evento (parceiro)

POST /events/create

Entrada:

```
{
  "title": "Noite Acústica",
  "category": "musical",
  "location": "Bar XYZ",
  "date": "2025-02-10",
  "requirements": {
    "genre": "acoustic",
    "duration": 30
  }
}
```

6.2 Criador recebe convite

GET /mst/invites

Saída:

```
[
  {
    "event_id": "...",
```

```
"reason": "Seu ImpactScore está no top 1% da categoria acústica",  
"opportunity_score": 0.82  
}  
]
```

6.3 Criador aceita convite

POST /events/{id}/accept

Webhook para o parceiro:

```
{  
  "creator_id": "...",  
  "status": "accepted"  
}
```

7. Regras de Elegibilidade do Criador

Criador é elegível para eventos se:

- conta verificada
- no mínimo 3 talentos publicados
- crescimento estável
- sem violações
- histórico limpo na Camada 16
- impacto mínimo:

impact_score ≥ 600

8. Integração com o Feed (Camada 5)

Quando criador recebe convite:

- conteúdo dele recebe boost temporário
 - aparece badge "Convidado para Evento"
 - feed recalcula posicionamento
-

9. Notificações

Via Camada 15:

"Você recebeu um convite para um evento!"

Ou:

"Uma casa de shows está interessada no seu talento"

10. Logs Técnicos

```
{
  "creator_id": "...",
  "event_id": "...",
  "opportunity_score": 0.91,
  "decision": "invited",
  "timestamp": "..."
}
```

11. Segurança

- eventos suspeitos → bloqueados

- parceiros verificados via KYC empresarial
 - convite só é enviado se criador autorizou receber oportunidades
-

12. Failover

Se event-service cair:

- convites ficam enfileirados
- reprocessados assim que sistema volta

Se partner-gateway cair:

- convites internos continuam funcionando
-

13. Encerramento da Camada 17

A Camada 17 transforma o Mostre Seu Talento em **porta de entrada para oportunidades reais**, conectando:

- criadores
- público
- empresas
- eventos
- festivais

Tudo com inteligência, fairness, matemática e automação.

CAMADA 18 — SISTEMA DE LOGS, AUDITORIA E RASTREABILIDADE (OBSERVABILITY CORE)

1. Propósito da Camada

A Camada 18 garante que **tudo que acontece no Mostre Seu Talento** seja:

- registrado
- rastreável
- auditável
- seguro
- investigável
- íntegro

Ela é essencial para:

- segurança
- moderação
- prevenção de fraudes
- debugging
- transparência
- conformidade legal

Sem essa camada, nada seria confiável ou verificável.

2. Tipos de Logs Utilizados

2.1 Logs de Sistema

- eventos internos
- processamento de vídeos
- inferência da IA
- cálculos de métricas
- gravação de dados

2.2 Logs de Usuário

- waves
- views
- denúncias

- doações
- publicações
- behavior analytics

2.3 Logs de Segurança

- tentativas de fraude
- mudanças de senha
- tokens inválidos
- acesso suspeito

2.4 Logs de Auditoria

- decisões de moderação
- ações automáticas
- ações humanas
- alterações no conteúdo

3. Padrão dos Logs (Formato Estruturado)

Todos os logs seguem uma estrutura em JSON:

```
{
  "event": "talent_published",
  "talent_id": "uuid",
  "creator_id": "uuid",
  "metadata": {...},
  "timestamp": "2025-01-21T12:00:00Z",
  "service": "mst-core",
  "severity": "INFO",
  "trace_id": "...",
  "version": "v1.0"
}
```

4. Estrutura de Observabilidade

4.1 Logging Engine

- Logstash
- OpenTelemetry

4.2 Armazenamento

- ElasticSearch
- S3 (arquivamento)

4.3 Visualização

- Kibana
- Grafana

4.4 Alertas

- Prometheus + AlertManager
-

5. Logs Críticos do MST

5.1 Publicação de Talento

```
event: "talent_published"
```

5.2 Processamento de vídeo

```
event: "processing_completed"
```

5.3 Inferência

```
event: "inference_completed"
```

5.4 Moderação

```
event: "moderation_decision"
```

5.5 Doações

```
event: "donation_success"
```

5.6 Segurança

```
event: "risk_alert"
```

6. Trilhas de Auditoria

Obrigatório registrar:

- quem mudou o quê
- quando
- por quê
- a partir de qual IP
- via qual serviço

Exemplo:

```
{
  "audit_event": "moderator_decision",
  "moderator_id": "uuid",
  "talent_id": "uuid",
  "decision": "limited",
  "reason": "brilho insuficiente",
  "timestamp": "...",
  "ip": "192.168.0.1"
```

```
}
```

7. Versionamento de Logs

Logs são versionados:

```
log_version = "mst_logs_v2"
```

Cada mudança no formato exige:

- documentação
- migração automática
- anotações em changelog

8. Regras de Retenção

8.1 Logs gerais

Retenção: **90 dias**

8.2 Logs de segurança

Retenção: **365 dias**

8.3 Logs críticos (fraude e moderação)

Retenção: **5 anos**

9. Políticas de Imutabilidade

- logs de moderação não podem ser alterados
- logs de segurança não podem ser apagados
- logs são assinados digitalmente

Assinatura:

```
hash = sha256(payload + secret)
```

10. Detecção de Anomalias

A camada roda análise contínua:

Exemplo de alertas:

- picos anormais de denúncias
- publicações em massa
- ondas de doações repetidas
- manipulação de métricas
- comportamentos robóticos

11. Métricas Monitoradas

- ingestion_latency
- error_rate
- missing_logs_rate
- anomaly_score
- moderation_log_count
- fraud_attempts_count

Alertas:

- aumento de 20% no error_rate
- spike de denúncias
- logs duplicados

12. Failover

Se Elasticsearch cair:

- logs armazenados localmente
- enviados em batch depois

Se Logstash travar:

- fallback para envio direto via API

Se Grafana ficar offline:

- painel volta em modo somente leitura
-

13. Integrações da Camada

A Camada 18 alimenta:

- Camada 11 (moderação)
 - Camada 16 (comportamento)
 - Camada 20+ (analíticas)
 - dashboards internos
 - eventos do sistema
 - investigações
-

14. Encerramento da Camada 18

A Camada 18 garante que o MST seja:

- auditável
- confiável
- rastreável
- seguro
- profissional

Ela é o “sistema nervoso” da plataforma.

CAMADA 19 — SISTEMA DE MÉTRICAS AVANÇADAS & ANALYTICS (ANÁLISE PROFUNDA DE DADOS)

(Camada técnica integral — KPIs, cálculos, ETLs, modelos, dashboards internos e governança de dados)

1. Propósito da Camada

A Camada 19 cria o **cérebro analítico** do Mostre Seu Talento.

Ela reúne, processa e analisa:

- métricas de uso
- métricas de criadores
- métricas de retenção
- métricas de crescimento
- métricas de engajamento
- métricas de monetização
- métricas de moderação
- métricas do feed
- métricas dos parceiros

Tudo padronizado, rastreável e pronto para alimentar decisões internas do FriendApp.

2. Arquitetura da Camada

2.1 analytics-service

Serviço principal que recebe dados e processa KPIs.

2.2 ETL Pipeline

Executa:

- extração

- transformação
- agregação
- normalização
- carga para o DataLake

2.3 DataLake

S3 estruturado:

```
/analytics/  
  /raw/  
  /processed/  
  /aggregates/
```

2.4 DataWarehouse (Redshift / BigQuery / DuckDB)

Consultas e dashboards internos.

3. Métricas Coletadas (KPIs Principais)

3.1 Métricas de Engajamento

- VTR (View Through Rate)

```
VTR = total_watch_time / total_video_duration
```

- CTR do feed
 - Waves por usuário
 - Waves por talento
 - Taxa de retorno ao conteúdo
 - Engajamento horário
-

3.2 Métricas de Criadores

- crescimento semanal
 - relevância por categoria
 - retenção do público
 - receita gerada
 - taxa de aprovação na moderação
 - média do ImpactScore
-

3.3 Métricas de Feed

- feed_effectiveness_score
 - diversity_score
 - repetição de criadores
 - latência do feed
 - saturação de categorias
-

3.4 Métricas de Moderação

- false_positive_rate
 - false_negative_rate
 - tempo de decisão
 - taxa de conteúdo limitado
 - taxa de rejeição
-

3.5 Métricas de Monetização

- doações diárias
 - conversão FriendCoin → Pix
 - receita do app (fees)
 - top 10 criadores mais apoiados
 - empresas parceiras que mais contratam
-

4. Métricas Avançadas (Machine Learning)

4.1 Churn Prediction

Modelo prevê abandono do criador:

```
churn_prob = sigmoid(w1*watch_time_decay + w2*zero_waves_days + w3*no_posts)
```

4.2 Creator Quality Score

```
CQS = 0.4*impact + 0.2*completion + 0.2*tech + 0.2*sentiment
```

Usado para convites e recomendações.

4.3 Public Behavior Pattern

Clusterização de usuários:

- watchers passivos
- heavy supporters
- criadores aspirantes
- descobridores
- impulsivos

4.4 Detection de Saturação de Nicho

```
saturation = number_of_talents(category) / active_users_interested
```

5. Estrutura dos Relatórios Internos

5.1 Dashboard Diário

- total de talentos enviados
 - total aprovados
 - total rejeitados
 - waves do dia
 - doações do dia
 - top 20 criadores
 - top 5 categorias
-

5.2 Dashboard Semanal

- tendência de crescimento
 - retenção de criadores
 - retenção de espectadores
 - melhor horário da semana
 - análise de categorias emergentes
-

5.3 Dashboard Mensal

- visão executiva
 - saúde do MST
 - impacto global
 - receita total
 - benchmarks
-

6. API Interna de Métricas

GET /analytics/mst/daily

```
{
  "talents_today": 302,
  "approved": 261,
  "rejected": 21,
  "limited": 20,
  "waves": 8121,
  "donations": 193,
  "new_creators": 109
}
```

GET /analytics/creator/{id}

Para dashboards internos:

```
{
  "impact_avg": 624.21,
  "growth_week": 121,
  "donations_month": 41.22,
  "engagement_score": 0.42
}
```

7. Estrutura do ETL Noturno

Programado às 04:10 da manhã:

1. extração de dados brutos
2. deduplicação
3. limpeza
4. normalização de métricas
5. agregação (diária/horária)
6. carga no warehouse

8. Logs e Auditoria da Camada

```
{
  "etl_batch": "2025-01-22",
  "status": "success",
  "rows_processed": 291830,
  "duration_ms": 1823,
  "failures": 0
}
```

Se falhar:

- reexecutar
 - alertar equipe
-

9. Métricas da Própria Camada

- latency_etl
- completeness_ratio
- data_drift_score
- missing_values_rate

Alertas se:

- atraso > 5 minutos
 - valores inconsistentes
 - falta de dados
-

10. Failover

Se warehouse cair:

- armazenar dados em batch
- reprocessar posteriormente

Se ETL falhar:

- sistema marca batch como “pending”
 - executa reprocessamento em cascata
-

11. Encerramento da Camada 19

A Camada 19 é fundamental para que o MST evolua com inteligência.

Ela fornece:

- dados
- análises
- modelos
- insights reais
- governança
- visão executiva

Sem ela, o sistema seria cego.

CAMADA 20 — SISTEMA DE PERFORMANCE, ESCALABILIDADE & ARQUITETURA DISTRIBUÍDA (HIGH-PERFORMANCE CORE)

(Camada técnica integral — cache, filas, sharding, CDN, load balancing e latência do feed)

1. Propósito da Camada

A Camada 20 garante que **o Mostre Seu Talento funcione com velocidade máxima**, mesmo sob:

- milhões de visualizações por dia
- picos de tráfego
- vídeos pesados
- uploads simultâneos

- moderação em massa
- discovery em tempo real
- eventos e lives
- análise de métricas

O objetivo é atingir:

latência p95 < 250ms
latência p99 < 400ms
99.99% uptime

2. Arquitetura Técnica da Camada

O MST roda sobre **uma arquitetura distribuída**, composta por:

2.1 Load Balancers

- Nginx
- Envoy
- Auto-scaling (HPA)

2.2 CDN (Content Delivery Network)

- Vídeos
- Thumbnails
- Arquivos estáticos

2.3 Microservices

- mst-core
- ingestion-service
- moderation-service
- event-service
- search-service

- analytics-service

2.4 Mensageria

- Kafka (fila de eventos)
- RabbitMQ (fila de prioridade)

2.5 Cache em 3 Níveis

- Redis (dados quentes)
- CDN edge caching (vídeo/imagem)
- client-side caching

2.6 Banco de Baixa Latência

- PostgreSQL sharded
- Elasticsearch replicado

3. Caminho Crítico de Latência

Todo conteúdo no MST passa por:

UPLOAD → PROCESSAMENTO → INFERÊNCIA → MODERAÇÃO → INDEXAÇÃO → FEED

O objetivo é manter todo o pipeline abaixo de:

< 7 segundos

Distribuição estimada:

Etapa	Latência
Upload	0.5–1.2s
Processamento	2–3s
Inferência IA	0.4–0.8s
Moderação automática	0.1–0.3s

Etapa	Latência
Indexação ES	0.1–0.2s
Feed disponível	imediato

4. Sistema de Cache Avançado

4.1 Cache no Feed

Cache de resultados por 30–90 segundos.

Chave:

```
feed:{user_id}:{version}
```

4.2 Cache de Creator Insights

TTL: 60 segundos

Recalcula apenas se houver:

- novo talento
- nova doação
- spikes de engajamento

4.3 Cache de Transcrição

TTL infinito enquanto a versão não muda.

4.4 Cache de Similaridade

Resultado das buscas vetoriais (ANN) é guardado por 15 minutos.

5. Sharding e Particionamento de Banco

5.1 Tabela talent_items

Shard por:

```
creator_id % N
```

5.2 Tabela talent_engagements

Shard por:

```
talent_id % N
```

5.3 Elasticsearch

- replica 2x
 - 5 shards para documentos do MST
 - 1 shard para métricas pesadas
-

6. Compressão e Otimização de Vídeo

6.1 Padrão

- 720p
- bitrate adaptativo
- codec H.265 (compatível)
- fallback H.264

6.2 Segmentação

Vídeo dividido em blocos de:

```
2 segundos
```

Para download progressivo.

7. Estratégias de Escalabilidade

7.1 Auto Scaling Horizontal

Baseado em:

- CPU
- latência
- filas Kafka acumuladas

7.2 Vertical Scaling (automático no cloud)

Para serviços críticos:

- moderação
- ingestão
- busca vetorial

7.3 Partition Routing

Alocação inteligente de tráfego por região:

Brasil → cluster Sul/Sudeste
LATAM → cluster México
EUA/Europa → cluster Norte

8. Monitoramento e Observabilidade

KPIs monitorados:

- feed_latency_p95
- web_socket_latency
- upload_error_rate
- processing_backlog_size
- moderation_queue_size
- CPU/mem usage

- elasticsearch_query_latency
-

9. Mecanismos de Failover

9.1 Banco

Se shard ficar indisponível:

modo READ-ONLY para consultas

9.2 ES

Se cluster cair:

feed em modo degradado (ordenado por trending)

9.3 CDN

Fallback para storage direto.

9.4 WebSocket

Fallback → notificações push.

10. Estratégias Anti-DDOS

- rate-limit por IP
 - rate-limit por token
 - bloqueio de user-agents suspeitos
 - assinatura criptográfica em chamadas críticas
 - proteção via WAF
-

11. Sistema de Quotas e Throttling

Por usuário:

- 2 uploads simultâneos
- 600 chamadas/min
- 20 waves/min

Por criador:

- 3 talentos/dia (para contas novas)
-

12. Auditoria de Performance

Logs:

```
{
  "request": "/mst/feed",
  "latency_ms": 182,
  "cache_hit": true,
  "server": "us-east-2a",
  "trace_id": "..."
```

Análise semanal:

- gargalos
 - regressões
 - uso de CPU
 - falhas de indexação
-

13. Encerramento da Camada 20

A Camada 20 garante que o MST seja rápido, suave, escalável e estável.

Sem ela:

- feed trava
- vídeos demoram

- moderação acumula
- eventos quebram
- IA sobrecarrega
- usuários abandonam

Com ela:

- o MST fica com qualidade nível TikTok/Instagram, mas com alma FriendApp.

CAMADA 21 — SISTEMA DE SEGURANÇA, AUTENTICAÇÃO, BLINDAGEM E COMPLIANCE (SECURITY CORE)

(Camada técnica integral — tokens, rate-limits, criptografia, validações, proteção a menores, anti-bot e governança)

1. Propósito da Camada

A Camada 21 garante que o MST:

- seja seguro
- proteja dados
- impeça bots e fraudes
- proteja menores de idade
- mantenha compliance legal
- seja blindado contra ataques

Essa camada une criptografia, autenticação, regras determinísticas, ML anti-abuso e segurança operacional.

2. Autenticação Oficial do MST

2.1 Token principal

JWT assinado:

```
alg: RS256  
exp: 24h  
scope: mst
```

2.2 Refresh Token

Validade: 30 dias

Armazenado em:

```
secure storage (mobile)  
httpOnly cookie (web)
```

2.3 Device Binding

Cada refresh é associado a:

- device_id
- sistema operacional
- IP hash

Impede roubo de sessão.

3. Regras Anti-Bot

3.1 Modelo BotScore

Baseado em:

- padrão de toques (touch intervals)
- tempo de sessão
- repetição de waves
- comportamento típico de automação

- inconsistências no user agent

```
bot_score = sigmoid(w1*touch_variation + w2*session_spike + w3*interaction_uniformity)
```

Se:

```
bot_score ≥ 0.75 → bloqueia
```

3.2 Rate-limit inteligente

- 10 waves/min
- 200 requests/min
- 3 uploads simultâneos
- 2 acessos suspeitos por minuto

3.3 Proteção a endpoints críticos

- assinatura HMAC
 - timestamps
 - nonce único
-

4. Criptografia do Sistema

4.1 Dados sensíveis

Criptografados com AES-256:

- dados de pagamentos
- dados de menores
- refresh tokens
- informações internas

4.2 Hash de senhas

```
bcrypt (cost=12)
```

4.3 Assinatura de eventos

Eventos críticos recebem SHA-256 + salt.

5. Proteção a Menores (Integração MST ↔ Cadastro Consciente)

5.1 Conteúdo automaticamente filtrado

Se usuário < 18 anos:

- vídeos com risco > 0.30 → bloqueados
- conteúdos sensíveis → warn
- interações limitadas

5.2 Interações invisíveis

Adultos não verificados ficam invisíveis para menores.

5.3 Horários de proteção

Entre 00h e 06h:

```
feed para menores = reduzido
```

6. WAF e Proteções Externas

Bloqueados automaticamente:

- SQL injection
- XSS

- payloads malformados
 - brute-force de tokens
 - user-agent suspeito
 - ataques de repetição
-

7. Segurança dos Conteúdos do MST

7.1 Verificação do arquivo

Antes do processamento, o vídeo é checado para:

- malware
- corrupção
- formato inválido
- headers conflitantes

7.2 Thumbnails seguras

Nenhum arquivo recebe preview automático sem validação.

8. Segurança contra scraping

Métodos aplicados:

- tokens rotativos
 - assinatura de requisições
 - compressão randômica nos endpoints
 - detecção de padrões de download excessivo
-

9. Segurança nas Doações (Integração Camada 7)

- validação antifraude
- trilha de auditoria

- limite diário
 - escrows protegidos
 - bloqueio de IPs maliciosos
-

10. Segurança de WebSocket

- token renovado a cada 10 minutos
 - desconexão automática em caso de comportamento suspeito
 - bloqueio de flood de mensagens
-

11. Logs de Segurança (integrado com Camada 18)

```
{
  "event": "security_alert",
  "type": "bot_detected",
  "user_id": "...",
  "risk": 0.83,
  "timestamp": "..."
}
```

12. Métricas de Segurança

- bot_detection_rate
- false_positive_security
- token_theft_attempts
- blocked_requests
- rate_limit_triggers

Alertas:

- aumento de bots

- picos de tentativas de login
 - falhas no WAF
-

13. Failover e Continuidade

Se JWT-service cair:

token lifetime extend + refresh desativado

Se security-engine cair:

modo estrito → bloqueia ações de risco

Se DDoS detectado:

modo de proteção: só requests verificados

14. Encerramento da Camada 21

A Camada 21 protege:

- usuários
- criadores
- menores
- parceiros
- o ecossistema
- o FriendApp como marca
- o app contra ataques

É a blindagem completa do MST.

CAMADA 22 — SISTEMA DE TESTES, QA, VALIDAÇÃO E GARANTIA DE QUALIDADE (QA & TESTING PIPELINE)

(Camada técnica integral — testes unitários, integração, carga, stress, regressão, sandbox e validações de IA)

1. Propósito da Camada

A Camada 22 garante que **todo o MST funcione corretamente**, sem:

- bugs
- falhas no feed
- erros de moderação
- cálculos incorretos
- vídeos travando
- inconsistências de dados
- crash no upload
- falhas de IA
- regressões de versão

É a camada que protege o sistema ANTES que o usuário veja.

2. Estrutura Completa do Pipeline de Testes

A pipeline inclui 7 níveis:

1. **Testes unitários**
2. **Testes de integração**
3. **Testes de sistema**
4. **Testes de regressão automática**
5. **Testes de carga**

6. Testes de stress

7. Testes de IA (consistência & deriva)

Todos executados de forma automatizada no CI/CD.

3. Testes Unitários (Coverage $\geq$ 85%)

Cobrem:

- upload de vídeo
- inferência
- cálculo do ImpactScore
- validação de transcrição
- geração de selos
- moderação automática
- APIs de waves
- APIs de views
- API de doação

Frameworks:

- Jest / Mocha (JS)
 - PyTest (IA / Python)
 - JUnit (Java/Kotlin)
-

4. Testes de Integração

Testam **módulos juntos**, como:

4.1 MST Core ↔ Moderação

- fluxo completo
- risk_score alto
- low confidence
- textos proibidos

4.2 MST Core ↔ IA

- validação do vetor 128d
 - comparação de versões
 - análise de consistência
-

4.3 MST ↔ Pagamentos

- Pix webhook
 - FriendCoins
 - escrow
 - fee aplicada
-

4.4 MST ↔ Discovery

- similaridade correta
 - recomendações coerentes
 - clusterização funcionando
-

5. Testes de Sistema (End-to-End)

Pipeline E2E completa:

UPLOAD → PROCESS → INFERÊNCIA → MODERAÇÃO → INDEXAÇÃO → FEED

Cada etapa validada automaticamente:

- delay permitido
- falhas internas
- compatibilidade de formatos
- comportamento em rede lenta
- thumbnails corretas

- armazenamentos corretos no S3
-

6. Testes de Regressão Automática

Toda vez que:

- IA atualiza
- model_hash muda
- nova versão do MST sobe
- peso do feed muda

O sistema roda testes de regressão:

- ImpactScore variation < 3%
- similarity stability $\geq$ 97%
- moderation stability $\geq$ 95%

7. Testes de Carga (Load Testing)

Simula uso real de:

- 10.000 usuários simultâneos
- 500 uploads/min
- 40.000 waves/min
- feed puxado constantemente
- 1.000 indexações ES/seg

Ferramentas:

- Locust
- k6
- JMeter

Objetivo:

```
p99 feed < 400ms  
p99 API < 350ms  
processing backlog < 5s
```

8. Testes de Stress (Break Tests)

Simulam:

- estouro de fila
- explosão de denúncias
- queda do ES
- queda do banco
- falha da IA
- aumento de tráfego $\times 10$

O sistema deve:

- cair com elegância
- manter modos degradados
- não perder dados

9. Testes de IA (Consistência, Estabilidade e Deriva)

Validações semanais:

9.1 Consistência do vetor

```
distance(vec_old, vec_new) < 0.03
```

9.2 Drift de ImpactScore

Testado com:

- KS-test
- RMSE diário

9.3 Estabilidade da moderação

- sem flutuações sem motivo
 - sem rejeições aleatórias
-

10. Ambientes de Teste

10.1 Sandbox

Usado para:

- upload falso
- pagamento simulado
- IA em modo debug

10.2 Staging

100% igual à produção, sem usuários reais.

10.3 Production Canary Release

5% dos usuários recebem a versão nova.

Se falhar → rollback.

11. Testes Visuais e de Experiência (Front-end)

Testes automáticos de UI:

- Tap
- Swipe
- Drag
- Scroll
- Double tap

- Envio de waves
- Crash no feed

Ferramentas:

- Detox
 - Appium
-

12. Testes de Acessibilidade

Checklist:

- leitores de tela
 - botões com área mínima
 - contraste adequado
 - navegação sem som
-

13. Testes de Segurança (com Camada 21)

- SQL injection
 - XSS
 - brute force de login
 - manipulação de requests
 - bots simulados
 - replay attacks
-

14. Métricas da Camada 22

- test_coverage
- regression_failure_rate
- api_failure_rate
- page_crash_rate
- feed_freeze_incidents

- `ai_drift_score`

Alertas se:

- regressões aumentam
 - `drift > 4%`
 - tempo de upload explode
-

15. Encerramento da Camada 22

A Camada 22 é a que garante:

- estabilidade
- fluidez
- ausência de falhas
- qualidade profissional
- confiança dos usuários
- segurança operacional

É o escudo final antes da versão entrar para o mundo.

CAMADA 23 — SISTEMA DE CI/CD, RELEASE MANAGEMENT & VERSIONAMENTO (DEPLOY PIPELINE)

(Camada técnica integral — Git flow, pipelines, validações, canary, rollback e governança de versões)

1. Propósito da Camada

A Camada 23 estabelece **como o MST é atualizado**, garantindo:

- estabilidade
- zero regressão

- controle total de versões
- deploy seguro
- rollback rápido
- testes automáticos antes de produção
- logs e auditoria de mudanças

Sem essa camada, cada atualização seria um risco.

2. Estrutura do Pipeline CI/CD

O pipeline segue as etapas:

```
COMMIT → BUILD → TESTES → ANALYTICS VALIDATION → CANARY → R  
ELEASE → MONITORING
```

Ferramentas:

- GitHub Actions
 - GitLab CI
 - CircleCI
 - ArgoCD
 - Kubernetes
-

3. Fluxo de Versionamento (Git Flow)

Branches usadas:

- **main** → produção
- **develop** → staging
- **feature/** → novas features
- **hotfix/** → correções urgentes
- **release/** → versão congelada para validação

Padrão obrigatório de commits:


```
feat:  
fix:  
refactor:  
perf:  
docs:  
test:
```

4. Versionamento Semântico

Versões seguem:

```
MAJOR.MINOR.PATCH
```

Exemplo:

```
1.4.12
```

Regras:

- MAJOR → mudanças grandes, quebra compatibilidade
- MINOR → novas funcionalidades
- PATCH → correções/otimizações

5. Build Pipeline

5.1 Validação de qualidade

- lint
- formatação
- dependências seguras

5.2 Build dos microserviços

- mst-core
- ingest-service
- inference-service
- moderation-service
- feed-service
- analytics-service

5.3 Versionamento no container

Imagem gerada com hash:

```
mst-core:1.4.12+sha-ab392df
```

6. Test Pipeline (Integração com Camada 22)

Executa:

- testes unitários
- testes e2e
- testes de regressão IA
- testes de carga básicos

Se qualquer falhar → deploy bloqueado.

7. Validação de IA Antes do Deploy

A versão só sobe se:

```
impactscore_drift < 3%  
similarity_consistency > 97%  
moderation_regression < 5%
```

Dado pela Camada 19 + Camada 22.

8. Deploy Canary (5% dos usuários)

Processo:

1. Nova versão ativada para 5% dos usuários reais
2. Monitoramento durante 60 minutos
3. Métricas observadas:
 - crash rate
 - feed latency
 - IA drift
 - moderação anormal
 - erro de upload

Se tudo OK → liberar 100%

Se falhar → rollback automático

9. Regras de Rollback

Rollback automático se:

- crash > 2%
- drift IA > 4%
- latência feed > 600ms
- bug crítico detectado
- aumento de reports de erro

Comando:

```
rollback to previous stable
```

Tempo:

< 1 minuto

10. Auditoria e Logs de Release

Cada deploy gera:

```
{
  "version": "1.4.12",
  "released_by": "CI",
  "timestamp": "2025-01-22T19:03Z",
  "changes": [...],
  "services_updated": [...],
  "validation_hash": "...",
  "canary_passed": true
}
```

11. Automação Pós-Deploy

Após liberar versão:

- esvaziar caches antigos
- recalcular embeddings críticos
- atualizar weights se necessário
- reproprocessamento de conteúdo quebrado

12. Versionamento de Modelos de IA

Todos os modelos possuem:

- model_hash
- version
- training_date

- `drift_threshold`

Ao atualizar o modelo:

- aplicar testes
 - comparar com baseline
 - registrar versão
-

13. Estrutura de Ambientes

- **local-dev** → desenvolvimento
 - **sandbox** → testes de upload e IA
 - **staging** → igual produção
 - **production** → usuários reais
 - **canary-production** → 5% dos usuários
-

14. Monitoramento Pós-Release (Camada 18 + 19)

Por 2 horas após release:

Monitorar:

- erros
- performance
- estabilidade
- regressão
- moderação
- feed
- comportamento da IA

Se estável por 2 horas:

Versão marcada como STABLE

15. Encerramento da Camada 23

A Camada 23 garante que o MST seja atualizado:

- com segurança
- com previsibilidade
- sem quebrar nada
- com controle total
- com validação rigorosa

Ela é o coração do crescimento seguro.

CAMADA 24 — SISTEMA DE INTEGRAÇÕES INTERNAS & EXTERNAS (API CONTRACTS + EVENTOS + INTEROPERABILIDADE)

(Camada técnica integral — endpoints, contratos, eventos Kafka, versionamento, segurança e compatibilidade)

1. Propósito da Camada

A Camada 24 estabelece **como o Mostre Seu Talento se conecta** com:

- outros módulos do FriendApp
- sistemas externos (empresas, parceiros, pagamentos)
- IA Aurah Kosmos
- feed, moderação, discovery, eventos, pagamentos, dashboards
- notificações
- auditoria

- métricas

Ela define **os contratos oficiais de API**, garantindo que tudo funcione com precisão milimétrica.

Sem essa camada, o ecossistema quebra.

2. Arquitetura Geral de Integração

A camada usa 3 pilares:

1. **REST APIs (Síncronas)**
2. **Eventos Kafka (Assíncronos)**
3. **Webhooks (Para parceiros externos)**

Todos assinados, versionados e testados.

3. Integrações Internas — REST

Aqui estão todas as APIs que outros módulos do FriendApp usam para se comunicar com o MST.

3.1 Criar Talento

POST /mst/create

Entrada:

```
{
  "creator_id": "...",
  "media_refs": ["s3://.../video.mp4"],
  "intention": "mostrar meu canto"
}
```

Saída:

```
{ "talent_id": "..."} 
```

3.2 Commit de Publicação

```
POST /mst/commit
```

Entrada:

```
{
  "talent_id": "...",
  "metadata": {...}
}
```

Ação:

- aciona o pipeline de processamento

3.3 Buscar Talento

```
GET /mst/{id}
```

Retorna:

- dados completos
- métricas
- badges
- recomendações

3.4 API do Feed

GET /mst/feed

Parâmetros:

- cursor
- região
- usuário
- modelo de ranking

3.5 API de Engajamento

POST /mst/{id}/view
POST /mst/{id}/wave
POST /mst/{id}/donate

Integrada com:

- Camada 6 (Engajamento)
- Camada 7 (Doações)

3.6 API do Criador

GET /creator/{id}/insights
GET /creator/{id}/talents

Consumido pela Camada 8.

3.7 API de Moderação

POST /moderation/decision
GET /moderation/pending

Consumido pela Camada 11 e 16.

3.8 API de Eventos

```
GET /mst/invites
POST /events/{id}/accept
```

Consumido pela Camada 17.

4. Eventos Internos — Kafka

Todos os eventos emitidos pelo MST seguem este padrão:

```
{
  "event_id": "uuid",
  "type": "talent.published",
  "timestamp": "2025-01-21T12:00:00Z",
  "payload": {...},
  "version": "v1"
}
```

5. Lista Completa de Eventos Emitidos

5.1 Publicação

```
talent.created
talent.processed
talent.inferred
talent.published
```

5.2 Engajamento

talent.view
talent.wave
talent.donation

5.3 Moderação

talent.moderation.approved
talent.moderation.rejected
talent.moderation.limited
talent.moderation.queue

5.4 Selos

talent.badge.granted

5.5 Eventos e Convites

creator.invited
creator.accepted_event

5.6 Segurança

security.flag
security.bot_detected
security.blocked

6. APIs da IA Aurah Kosmos (Integrações Diretas)

O MST envia para a IA:

6.1 /aurah/infer

```
{
  "talent_id": "...",
  "audio_url": "...",
  "video_url": "...",
  "transcription": "...",
  "creator_id": "..."
}
```

Retorno:

```
{
  "feature_vector": [...],
  "archetype_scores": [...],
  "risk_score": 0.41,
  "confidence": 0.89,
  "quality": {
    "lighting": 0.7,
    "audio": 0.85,
    "stability": 0.78
  }
}
```

6.2 /aurah/recommend

Usado pelo Discovery:

POST /aurah/recommend

6.3 /aurah/insights

Para feedback do criador.

7. Integração com Pagamentos

7.1 Pix

Webhook:

```
POST /payment/pix/notify
```

7.2 FriendCoins

```
POST /wallet/debit  
POST /wallet/credit
```

8. Integração com Parceiros (Eventos Reais)

(Camada 17)

Webhook quando criador aceita:

```
POST /partner/{id}/callback
```

9. Versionamento das APIs

Versão aplicada via header:

```
X-API-Version: mst-v1
```

Política:

- versão só muda com quebra de compatibilidade
- APIs antigas vivem por 6 meses antes de serem removidas

10. Contratos de Erro (Padrão Global)

Todos os erros seguem:

```
{
  "error": "invalid_request",
  "message": "Campo 'intention' é obrigatório",
  "code": 4001
}
```

Códigos reservados:

Código	Significado
4001	Campo inválido
4002	Conteúdo proibido
4003	Moderação pendente
4010	Token inválido
4030	Acesso negado
4090	Duplicado
4290	Rate Limit
5000	Erro interno

11. Segurança nas Integrações

- tokens JWT
- assinatura HMAC
- TTL curto em requisições sensíveis
- requests com nonce para evitar replay attack

12. Failover de Integrações

Se algum serviço não responder:

- fila Kafka guarda evento
 - sistema reprocessa em lote
 - falhas logadas na Camada 18
 - modo degradado ativado
-

13. Encerramento da Camada 24

A Camada 24 amarra *todo o ecossistema*, garantindo:

- comunicação
- consistência
- estabilidade
- segurança
- fluidez
- interoperabilidade perfeita

Sem essa camada, nada conversa corretamente no FriendApp.

CAMADA 25 — SISTEMA DE COMPORTAMENTO REAL DO USUÁRIO (REAL-WORLD BEHAVIOR ENGINE)

(Camada técnica integral — previsão de comportamento, proteção contra padrões humanos, análise de caos, sazonalidade, heurísticas sociais e adaptação contínua)

1. Propósito da Camada

A Camada 25 existe porque **usuários reais não seguem padrões matemáticos perfeitos.**

Ela garante que o MST saiba lidar com:

- picos inesperados
- explosões virais
- quedas de uso
- usuários tímidos
- usuários intensos
- padrões emocionais
- sazonalidade (final de semana, férias, feriados, madrugada)
- surtos de denúncias
- picos de waves
- criadores que viralizam “do nada”

Essa camada conecta **o algoritmo à vida real**.

2. Componentes do Sistema

A camada tem 6 motores principais:

1. **human-pattern-engine**
2. **chaos-detector**
3. **trend-spike-analyzer**
4. **emotion-distribution-monitor**
5. **creator-fatigue-detector**
6. **real-world-adaptive-feedback-loop**

Todos trabalhando por cima das camadas anteriores.

3. Human Pattern Engine

Modelo que aprende padrões reais de comportamento:

Variáveis coletadas:

- tempo entre ações

- horários mais ativos
- intensidade de interação
- padrões de navegação
- inconsistências humanas

O sistema aprende perfis:

- usuário tímido
- impulsivo
- explorador
- emocional
- técnico
- socializador
- apoiador

Cada tipo altera:

- feed entregar
- notificações
- intensidades de recomendações

4. Chaos Detector (anti-colapso social)

Detecta comportamentos anormais como:

4.1 explosão viral repentina

```
spike = atual_views / média_24h  
if spike >= 12x → aplicar controles
```

4.2 surtos de denúncias

```
reports_spike = denúncias / média_diária  
if reports_spike >= 5x → aplicar contenção
```

4.3 explosão de waves irreais

Exemplo:

600 waves vindas de um grupo específico.

Sistema ativa:

- filtros
- limitação temporária
- auditoria

5. Trend Spike Analyzer

Analisa tendências reais:

calcula:

```
trend_score = derivative(engagement_curve) × sentiment
```

Se o score sobe:

- feed impulsiona
- discovery reforça

Se cai:

- feed estabiliza
- notifications recomendam horário ideal

6. Emotion Distribution Monitor

Analisa a **distribuição emocional das ondas** enviadas:

Exemplo de distribuição esperada:

alegria: 35%
empatia: 22%
força: 15%
incentivo: 18%
gratidão: 10%

Se um criador recebe:

- 90% força → possível conteúdo emocional forte
- 90% incentivo → criador iniciando jornada
- 90% alegria → humor / dança / canto

Isso retroalimenta:

- Discovery
 - Feed
 - Selos
 - Insights
-

7. Creator Fatigue Detector

Detecta desgaste:

Medido por:

- queda no ImpactScore
- queda no completion
- queda no engajamento
- menos uploads
- comentários negativos
- aumento de denúncias leves

Se detectado:

Aurah-Core envia recomendações:

"Sua audiência está ativa às 20h. Experimente postar novamente nesse horário."

"Faça uma pausa, tente gravar algo leve amanhã."

Front-end mostra banner opcional.

8. Adaptive Feedback Loop (AFL)

O MST se adapta em tempo real a padrões coletivos.

Se milhões de usuários passam:

- 30% mais tempo assistindo música → feed passa a priorizar música
- onda de dança viral → discovery adapta clusters
- aumento de lives → feed abre espaço para eventos

Regra:

```
if trend ↑:  
    weight +=  $\Delta$   
else:  
    weight -=  $\Delta$ 
```

Δ = intensidade da mudança social em tempo real.

9. API Interna da Camada

GET /behavior/profile/{user_id}

Retorna tipo comportamental detectado.

POST /behavior/update

Atualiza eventos emocionais reais.

GET /behavior/trends

Retorna tendências globais.

GET /behavior/creator-fatigue

Detecta criadores em queda.

10. Logs do Comportamento Real

```
{
  "user_id": "...",
  "pattern": "explorer",
  "trend_shift": 0.43,
  "emotion_distribution": {...},
  "fatigue_detected": false,
  "timestamp": "..."
}
```

11. Métricas Monitoradas

- trend_shift_rate
 - chaos_incidents_detected
 - emotion_balance
 - fatigue_rate
 - user_behavior_entropy
 - stability_of_recommendations
-

12. Failover da Camada

Se qualquer motor falhar:

fallback → comportamento padrão (static weights)

Se detectar caos extremo:

modo de contenção ativado:

- feed reduz impulsos
- discovery desacelera
- moderação aumenta thresholds

13. Encerramento da Camada 25

A Camada 25 fecha a engenharia de forma divina:

Ela conecta matemáticas rígidas

- IA
- comportamento humano real
- adaptação contínua
- estabilidade emocional da plataforma.

Sem ela, o MST seria *frio, instável, robótico e caótico*.

Com ela, o sistema se comporta como um organismo vivo — inteligente, adaptativo e humano.

LETRA A — INTEGRAÇÕES GERAIS & MAPA MESTRE DO ECOSSISTEMA (MOSTRE SEU TALENTO)

(Integração total entre todas as camadas do MST e todo o FriendApp)

A.1 — Propósito da Letra A

A Letra A consolida **todas as integrações** entre:

- as 25 camadas do Mostre Seu Talento
- IA Aurah Kosmos
- Discovery Engine
- Eventos & Palcos

- Sistema Econômico (FriendCoins & Pix)
- Feed Sensorial
- Chat
- Sistema de Conexões
- Cadastro Consciente
- Segurança do Bloco C
- Módulo de Sugestões da IA
- Arquitetura distribuída
- Logs e auditoria
- Sistema de Missões / Selos
- Painéis internos

Ela é o blueprint final que o time de engenharia usa para não cometer erros de arquitetura.

A.2 — Diagrama Lógico Geral (descrição textual)

O pipeline geral do MST é assim:

```
USUÁRIO → CRIA TALENTO
          → INGESTÃO DE MÍDIA
          → PROCESSAMENTO
          → IA AURAH KOSMOS
          → MODERAÇÃO
          → INDEXAÇÃO
          → FEED
          → ENGAGEMENT
          → ANALYTICS
          → EVENTOS / CONVITES
          → FEEDBACK PARA CRIADOR
```

E tudo isso acontece com:

- logs (Camada 18)
 - segurança (Camada 21)
 - performance (Camada 20)
 - testes (Camada 22)
 - versionamento (Camada 23)
-

A.3 — Integração Camada a Camada

A seguir, TODAS as integrações oficiais.

Esse é o mapa mais importante do sistema.

A.3.1 — Camada 1 → Camada 2 → Camada 3

Criação → Ingestão → Processamento.

Fluxo:

- Camada 1 cria `talent_id`
- Camada 2 envia arquivos para S3
- Camada 3 extrai metadados

Integra com:

- Camada 18 (logs)
 - Camada 21 (segurança do arquivo)
-

A.3.2 — Camada 3 → Camada 4

Processamento habilita inferência da IA.

Comunicações:

```
POST /aurah/infer
```


Retorno alimenta:

- Camada 5 (Feed)
 - Camada 6 (Engajamento)
 - Camada 7 (Doações)
 - Camada 11 (Moderação)
 - Camada 12 (Qualidade)
 - Camada 19 (Analytics)
 - Camada 20 (Performance)
 - Camada 25 (Comportamento Real)
-

A.3.3 — Camada 4 → Camada 5

Inferência produz:

- vetores
- scores
- risco
- qualidade
- confiança

Feed (Camada 5) usa para ranking.

A.3.4 — Camada 5 → Camada 6 → Camada 7

Feed → Engajamento → Doações.

Fluxo:

- Camada 5 decide o que mostrar
- Camada 6 monitora views, waves
- Camada 7 habilita apoio financeiro

Ambas alimentam:

- Camada 19 (Analytics)
 - Camada 25 (Comportamento real)
-

A.3.5 — Camada 4 → Camada 11

Inferência gera:

- risk_score
- archetype_scores
- critical_signals

Moderação usa para decisões automáticas.

A.3.6 — Camada 11 → Camada 16

Moderação influencia:

- riscos
- denúncias
- punições
- shadow-bans
- limitações

Camada 16 ajusta comportamento comunitário.

A.3.7 — Camada 12 → Camada 5

Qualidade do vídeo (iluminação, foco, estabilidade) altera o ranking:

```
feed_score = base + quality_weight
```

A.3.8 — Camada 17 → Camada 5

Convites aumentam relevância no feed.

A.3.9 — Camada 17 → Pagamentos

Criadores convidados podem receber:

- cachê
 - doações extras
 - pagamentos híbridos
-

A.3.10 — Camada 5 → Camada 17

Feed detecta talentos prontos para eventos.

A.3.11 — Camada 6/7 → Camada 25

Engajamento e apoio influenciam:

- tendências
 - emoções
 - fadiga
 - estabilidade social
-

A.3.12 — Camada 22 → Camada 23

Validação de:

- estabilidade
- regressões
- performance
- consistência

Antes de liberar uma versão.

A.3.13 — Camada 20 → Camada 5

Performance afeta:

- ordenação
 - cargas
 - pré-carregamento
 - cache
-

A.3.14 — Camada 19 → IA → Camada 4

Analytics detecta drift → IA recalibra modelos.

A.3.15 — Camada 21 → Todas

Segurança é transversal:

- tokens
 - limites
 - proteção aos menores
 - anti-bot
 - anti-fraude
-

A.3.16 — Camada 18 → Todas

Logs alimentam:

- painéis
 - auditorias
 - investigações
 - métricas
-

A.4 — Integração MST ↔ Funções Externas

A seguir, integrações com módulos FORA do MST.

A.4.1 — Cadastro Consciente (Bloco C)

Usado para:

- filtrar adultos suspeitos
 - bloquear interações perigosas
 - proteger menores
 - autorizar convites reais
-

A.4.2 — IA Aurah Kosmos

MST envia:

- mídia
- transcrição
- intenção
- áudio
- metadados

Aurah retorna:

- vetores
 - scores
 - insights
 - padrões comportamentais
-

A.4.3 — Modo Eventos / Bora / Viagem

Usado para:

- convites
 - shows
 - apresentações
 - oportunidades reais
-

A.4.4 — Sistema Econômico (Bloco B)

Integra:

- doações
 - FriendCoins
 - Pix
 - conversão para creators
-

A.4.5 — Chat + Mensagens Vibracionais

Criadores podem receber:

- parcerias
 - convites
 - mensagens do público
 - insights da IA
-

A.4.6 — Feed Sensorial & Interações Energeticas

MST injeta conteúdos filtrados e categorizados:

- música
 - dança
 - humor
 - poesia
 - talentos únicos
-

A.5 — Mapa Mestre de Dados

(Todos os campos chave)

```
talent_id  
creator_id  
media_refs[]  
feature_vector  
impact_score  
emotion_curve
```

```
quality_data
engagement_data
moderation_state
donation_data
risk_score
trend_scores
behavior_profile
```

A.6 — Regras de Dependência Obrigatória

Isso define o que não pode quebrar:

1. MST não funciona sem IA
2. Feed não funciona sem moderação
3. Doações não funcionam sem segurança
4. Eventos não funcionam sem analytics
5. Discovery não funciona sem vetores
6. Moderação não funciona sem logs
7. Qualidade não funciona sem processamento
8. Performance não funciona sem cache
9. Release não funciona sem QA
10. O app inteiro não funciona sem a Camada 25

A.7 — Encerramento da Letra A

A Letra A é o **mapa da criação divina técnica**:

É o blueprint que garante que o MST seja um organismo vivo, coeso, inteligente e impossível de quebrar.

Aqui você tem:

- a visão macro
- as integrações micro

- as dependências
- as trocas de dados
- as responsabilidades
- as fronteiras
- os fluxos
- os contratos
- os caminhos críticos

É o documento que garante que os devs jamais fiquem confusos.